COMPUTER SCIENCE S5

STUDENT'S BOOK

Copyright

© 2022 Rwanda Basic Education Board

All rights reserved.

This book is the property of the Government of Rwanda.

Credit should be given to REB when the source of this book is quoted

FOREWORD

Dear Student,

Rwanda Basic Education Board is honoured to present to you this Computer Science book for Senior five which serves as a guide to competence-based teaching and learning to ensure consistency and coherence in the learning of Computer Science subject. The Rwandan educational philosophy is to ensure that you achieve full potential at every level of education which will prepare you to be well integrated in society and exploit employment opportunities.

The government of Rwanda emphasizes the importance of aligning teaching and learning materials with the syllabus to facilitate your learning process. Many factors influence what you learn, how well you learn and the competences you acquire. Those factors include the instructional materials available among others. Special attention to the activities that facilitate the learning process in which you can develop your ideas and make new discoveries during concrete activities carried out individually or with peers.

In competence-based curriculum, learning is considered as a process of active building and developing knowledge and meanings by the learner where concepts are mainly introduced by an activity, a situation or a scenario that helps the learner to construct knowledge, develop skills and acquire positive attitudes and values. For effective use of this textbook, your role is to:

- Work on given activities which lead to the development of skills
- Share relevant information with other learners through presentations, discussions, group work and other active learning techniques such as role play, case studies, investigation and research in the library, from the internet or from your community;
- Participate and take responsibility for your own learning;
- Draw conclusions based on the findings from the learning activities.

I wish to sincerely extend my appreciation to the people who contributed towards the development of this book, particularly REB staff who organized the whole process from its inception. Special gratitude goes to the University of Rwanda which provided experts in design and layout services, illustrations and image anti-plagiarism, lecturers and teachers who diligently worked to successful completion of this book. Any comment or contribution would be welcome for the improvement of this textbook for the next edition.

Dr. MBARUSHIMANA Nelson

Director General, REB

ACKNOWLEDGEMENT

I wish to express my appreciation to all the people who played a major role in development of this textbook. It would not have been successful without active participation of different education stakeholders.

I owe gratitude to different Universities and schools in Rwanda that allowed their staff to work with REB in the in-house textbooks production project. I wish to extend my sincere gratitude to lecturers, teachers and all other individuals whose efforts in one way or the other contributed to the success of writing of this textbook.

Special acknowledgement goes to the University of Rwanda which provided experts in design and layout services, illustrations and image anti-plagiarism..

Finally, my word of gratitude goes to the Rwanda Education Board staff particularly those from the Curriculum, Teaching and Learning Resources department (CTLR) who were involved in the whole process of in-house textbook writing.

Murungi Joan,



Head of Curriculum, Teaching and Learning Resources Department

TABLE OF CONTENTS

FOREWORD	**iii**
ACKNOWLEDGEMENT	**iv**
UNIT1: FUNDAMENTALS OF LAPTOP AND PORTABLE DEVICES	**3**
1.1. PORTABLE DEVICES	3
1.2 LAPTOP COMPUTER FUNDAMENTALS	7
1.3. Compare and contrast laptop and desktop components	11
1.4. SMARTPHONES, PDAs, TABLETS	12
1.5. LAPTOP AND PORTABLE DEVICES CLEANING PROCEDURES	15
END OF UNIT ASSESSMENT ACTIVITY	17
UNIT2: COMPLEX DATA STRUCTURE AND ALGORITHM	**21**
2.1. Introduction to Data Structures	22
2.2 Array of elements in data structure	23
2.3. ABSTRACT DATA STRUCTURE	31
2.4. List	31
2.5. Queue	40
2.6 Stack	43
2.7 Tree	46
2.8 Searching and sorting using complex data structure.	58
2.9 Sorting using complex data structure	61
2.10 Apply algorithm to solve complex mathematical functions	65
2.11 END UNIT ASSESSMENT	72
UNIT 3: INTRODUCTION TO COMPUTER NETWORK	**77**
3.1. Fundamental of computer Networ	78
3.2 Computer Network Concept and Technology	82
3.3 Local Area Networks (LANs)	86
3.4. Physical Components	89
3.5. Network Devices	93
3.6 Network Transmission Medium	95
3.7. IP Address	100
3.8. Data and device sharing	108
3.9. Network Topology	111

UNIT 4: INTRODUCTION TO DATABASE **119**
4.1. Definitions of key terms 119
4.2. Different area where database can be applied 121
4.3. Database approaches 121
4.4 Database access levels and users 126
END OF UNIT ASSESSMENT 134

UNIT 5: DATABASE DESIGN **137**
5.1 INTRODUCTION 137
5.2. DATABASE DESIGN STEPS 141
5.3. RELATIONAL MODEL 144
5.4. ENTITY-RELATIONSHIP MODEL 155
5.5. Database optimization through normalization 171
END UNIT ASSESSMENT 181

UNIT 6: POINTERS AND STRUCTURE IN C++ **185**
6.1 Pointers in C++ 186
6.2. Memory allocation 199
6.3 C++ Structures 203
END UNIT ASSESSMENT 214

UNIT 7: OBJECT ORIENTED PROGRAMMING IN C++ **217**
7.1. Introduction to Object Oriented Programming (OOP) 218
7.2 Class definition in C++ 221
7.3 Object in C++ 228
7.4 Data Encapsulation and Data Abstraction in C++ 236
7.5 Friend function in C++ 239
7.6 Polymorphism in C++ 242
7.7 Overloading 243
7.8 Constructors and Destructors 249
7.9. Inheritance in C++ 257
END OF UNIT ASSESSMENT 274

UNIT8: Introduction to Visual basic **281**
8.1. Understanding Visual Basic. 282
8.2 Visual Basic Standard EXE Integrated Development Environment (VB-IDE) 287
8.3 Visual basic controls 292
8.4. Planning and Developing a Visual Basic program.8 308

8.5 WORKING WITH MENUS AND DIALOG BOXES 320

UNIT 9: VARIABLES, OPERATORS, EXPRESSIONS AND CONTROL STRUCTURES. 329

9.1. Types of Visual Basic Data 330
9.2 Variables 333
9.3 Scope of a variable 340
9.4 Operators and expressions in Visual Basic 343
9.5. Decision structures in Visual Basic 351
9.6 Repetition structures 361
END UNIT ASSESSMENT 373

UNIT 10: INTRODUCTION TO JAVA 377

10.1. Concepts of Java Programming 377
10.2. Platforms of JAVA Program 379
10.3 Write, Compile and run a java Program 381
10.4. Compile the Source File into a .class File 385
10.5 Running the Program 386
10.6. Elements of Java source file 396
10.7 Flow control 401
END UNIT ASSESSMENT 408

UNIT 11: OOP AND JAVA 411

11.1. Classes vs Objects in Java 412
11. 2 Constructor in Java 416
11.3. The use of "new" and "This" keywords in java 421
11.4. Arrays in Java 429
11.5. Inheritance in Java 434
11.6 Access Modifiers in java 439
11.7 Overloading and overriding Method in Java 443
11.8 Encapsulation in Java 451
11.9 Abstract class in Java 455
END OF UNIT ASSESSMENT 460

UNIT 12: IO and Java 465

12.1. Introduction to IO Streams 465
12.2. InputStream, OutputStream and FileStream 466
12.3 FileStreams 473

12.4 Readers and writer 476
12.5 Character stream 478
END OF UNIT ASSESSMENT 481

BIBLIOGRAPHY **483**

UNIT 1
FUNDAMENTALS OF LAPTOP AND PORTABLE DEVICES

UNIT1: FUNDAMENTALS OF LAPTOP AND PORTABLE DEVICES

Key Unit Competency: To be able to identify, use and maintain safely laptops and other portable devices.

INTRODUCTORY ACTIVITY

Apply the following scenario:
You are in computer lab with different devices on table.



Figure 1.1. Mobile devices

Answer the following questions:

1. Name the devices you see on your table
2. Describe parts of each device.
3. Describe the common use and functions of these devices

1.1. PORTABLE DEVICES

Learning Activity 1.1

1) You are sited in your computer lab, observe well each device and then answer following questions:

a. Describe the portable devices available in lab;
b. Discuss the similarities and differences between available devices;
c. Enumerate other portable devices which are not in your lab.

2) In supermarket A, a laptop costs 350,000 Rwf while in supermarket B, laptop of same quality costs 300,000Rwf. Select where you will prefer to buy your laptop and explain why.

1.1.1. Portable devices definition

A *portable device* is a device that can easily be carried. It is a small form of a computing *device* that is designed to be held and used in the hands.

A portable device may also be called a handheld device or mobile device. Portable devices are primarily battery powered devices with base computing resources in the form of a processor, memory, storage and network access. Portable devices are becoming an increasingly important part of personal computing as the capabilities of devices like laptops, tablets and smartphones continue to improve.

11.2. Types of portable devices

a. Laptops

A laptop computer is a portable personal computer powered by a battery, or an Alternating Current (A.C) cord plugged into an electrical outlet, which is also used to charge the battery. Laptops have an attached keyboard and a touchpad, trackball, or isometric joystick used for navigation. Sometimes, a wireless mouse can also be used on a laptop. A wireless mouse is a computer mouse that needs no wires to send signals from the mouse to a computer. Wireless mouse technology predominantly uses radio frequencies (RF) to send signals from the mouse to the computer. Like other radio technologies, this requires a transmitter and a receiver. The mouse transmits radio signals to a receiver, which is itself connected to the computer hardware, normally via a wire. This kind of wireless mouse is very reliable, and capable of transmitting the mouse's movements to the receiver from across a room. A laptop also has a thin display screen that is attached and can be folded flat for transport. Laptop computer is also known as Notebook computer or simply Notebook.

The memory and storage capacity of laptop computer is almost equivalent to the PC or desktop computer. It also has the hard disk, DVD/CD optical disk drive, different connectors etc.



Figure 1.2. Laptop

b. Smartphones

A smart phone is a handheld personal computer with a mobile operating system and an integrated mobile broadband network connection for voice, SMS, and Internet data communication. Most if not all smartphones also support Wireless Fidelity (Wi-Fi). Smartphones are typically pocket-sized, as opposed to tablets, which are much larger. A smartphone is a cellular phone that performs most of the functions of a computer.

A smart phone supports all features of PDA plus mobile phone features

Smartphone characteristics/features

- Media players
- Internet access
- Global Positioning System(GPS)
- Wireless Fidelity (Wi-Fi)
- Camera
- Sensor

c. Mobile phones (cell phones)

A mobile phone or cell phone (is a long-range, electronic device used for mobile telecommunications (mobile telephony, text messaging or data transmission) over a cellular network. Today, this type of portable device is being dominated by smart phone as technology improves.



Figure 1.3: Mobile phones or cell phones

d. PDAs

A Personal Digital Assistant (PDA), also known as a palmtop computer, or Personal Data Assistant, is a mobile device that functions as a personal information manager. Newer PDAs commonly have color screens, audio capabilities and web browsers that enable them to be used as mobile phones (smartphones) or portable media players.

Many PDAs can access the Internet, intranets (communication within LAN) or extranets via Wi-Fi, or Wireless Wide Area Networks. Many PDAs employ touch screen technology.

e. Tablets

A tablet computer, or simply tablet, is a mobile computer with a large screen, circuitry and battery in a single unit with no built in keyboard. Some tablets include detachable keyboards have been sold since the mid-1990s. Such tablets include iPad, HP Touchpad, Android)

Tablets are very similar to smartphones. They have an Internet connection via a wireless connection including 3G/4G. Applications can be downloaded and installed. Sensors including speech recognition.



Figure 1.4: IPad

Application Activity 1.1.

6. Describe the difference between PDA and Smartphone
7. Compare a smart phone and Tablet
8. According to the comparison between laptop, Smart phone, Cell phone, PDA and Tablet, which one is most useful? Explain why.

1.2 LAPTOP COMPUTER FUNDAMENTALS

Learning Activity 1.2.

1) In the computer lab observe laptop computers and make groups and answer the following questions:

a) Discuss common uses of laptop computer in schools and society in general

b) Describe the external components of a laptop

c) Describe the internal components of a laptop

d) Investigate why laptop has Fn key on keyboard while desktop keyboard does not have it

e) With your laptops connected to Internet, search for steps to be followed while applying for National ID at www.irembo.gov.rw site

f) In the computer lab, identify the following features of a laptop: AC power port, ethernet port VGA port, vents, HDMI port, USB ports, camera.

g) Open a laptop and identify all its internal components

1.2.1. Common uses

The first laptops were used primarily by business people who needed to access and enter data when they were away from the office. The use of laptops was limited due to expense, weight, and limited capabilities compared to less expensive desktops.

Some common uses for laptops

Laptop is used in:

- Education field (example: Laptops are used by teachers in preparation of teaching and learning materials)
- Medicine field (example: Laptops are used by doctors to keep patient records)
- Business field (Electronic commerce uses Laptops for example in marketing and selling products or good and services.)
- Accessing online services (example: National ID application at www.irembo.gov.rw)
- Entertainment field (example: Playing games and watching movies while traveling)
- Communication field (example: when you send and receive an email in a public place)

- Security field (example: Laptop can be used to fight against hackers: people who access personal information without authorization of the owner. Notice that accessing of personal information without permission is punishable crime).

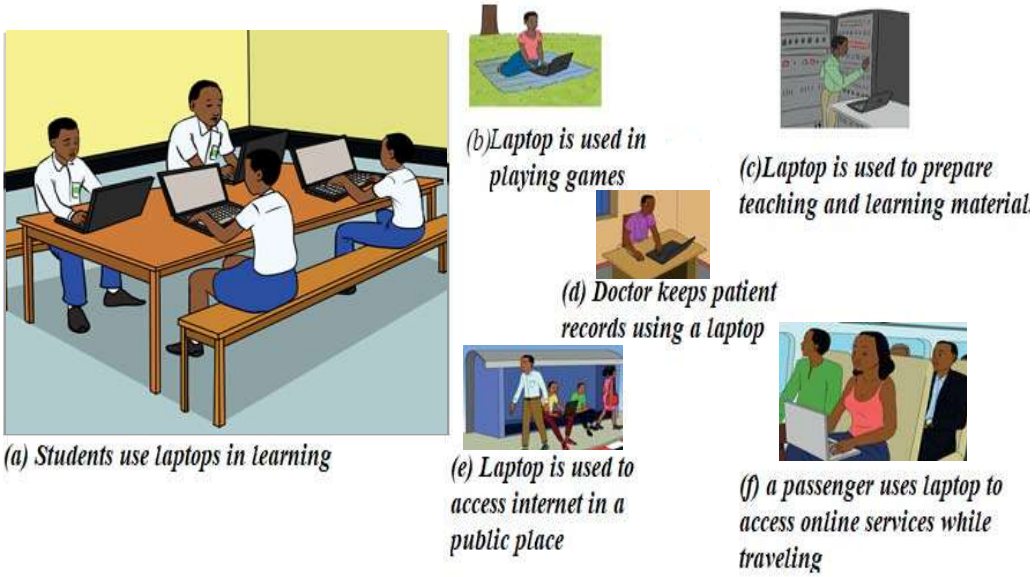


Figure 1.5. Graphical representation of some common uses of laptop

1.2.2. Laptop features

The most significant feature of a laptop is its compact size. The design of the laptop places the keyboard, screen, and internal components into a small portable case.

Common Characteristics/features of Laptops:

- Small and portable;
- An integrated display screen in the lid;
- An integrated keyboard in the base;
- They run on AC power or a rechargeable battery;
- Laptops support hot-swappable drives and peripherals.

1.2.3. Evolution (History) of laptops

In 1983, Gavilan SC is the first machine marketed as a "laptop"

In 1990, Intel introduces 80386SL processor which uses low power; quickly incorporated into many laptops.

In 1991, Apple PowerBook introduces First laptop Ethernet port. Modem speeds reach 14,400 bits/sec.

In 1996, First Pentium laptop with USB, first "three spindle" machine, with floppy,

hard disk, CD-ROM.

In 1999, Laptop manufacturers use PCI slot to enable Wi-Fi connectivity. Apple brands WiFi as AirPort, incorporates a slot into all of its laptops.

In 2005, Laptop computers outsell desktops for the first time.

In 2010, HDMI ports become common on laptops. Apple iPad was introduced.

The components of a laptop

Components may be located in different places for different models.

a. The external components of a laptop

The Laptop and the desktop computers use the same types of ports so that peripherals can be interchangeable. Ports, connections, and drives are located on the front, back, and sides of the laptop due to the compact design. There are status indicators, ports, slots, connectors, bays, jacks, vents, and a keyhole on the exterior of the laptop.

As port a laptop has an AC power port, an ethernet port, a VGA port a HDMI port, a USB port a sound in (microphone) port and a sound out (speaker) port. Beside these ports a laptop has a battery latch at its bottom

On the user view, which is normally visible when a laptop is being used one can see the following components:

1) Web camera
2) LCD
3) Power button
4) Keyboard
5) Touch pad
6) Right Click button
7) Left Click button



Figure 1.5. Laptop- User view

Note:

- The location of the power button may depend on the mark of the laptop
- Some laptops may not have an in-buitl CD/DVD disk drive while in other types, the CD/DVD disk drive can be located on left or right side. For those with no such drive an external drive that can be connected to the laptop through the USB port is often used.
- A desktop monitor can be connected to a laptop. In such cases, using the function key can allow one to toggle the screen from the laptop to the desktop monitor or view both at the same time.

b. The internal components of a laptop

The central circuit board (motherboard) makes up a complex electronic system. A motherboard provides the electrical connections by which the other components of the system communicate. It includes many components such as Central Processing Unit (CPU), Random Access memory (RAM), etc. Laptop motherboards and other internal components installation vary by manufacturer and are proprietary.

Internally and at the top a laptop after removing te cover there is access to the motherboard, speakers, cooling fans, wireless card, CMOS battery and audio board while under the bottom cover there is access to the hard drive and RAM module but this configuration may change from laptop mark to another

Application Activity 1.2

1) Suppose that your computer laptop is not connected to Internet. If there is no wireless connection,

a) which of the following cables can be used to have the connection?

i. VGA (Video Graphic Array) cable;

ii. Network cable;

iii. AC Power cable.

b) Describe the role of the remaining cables.

2) Choose from a laptop in the computer lab one document in PowerPoint and project it by using the Projector of your school.

3) Connect a desktop monitor to a laptop computer and then do the following:

i. Display the contents from the laptop to both screens;

ii. Display the contents from the laptop to only desktop monitor;

Differentiate HDMI port from USB port.

Open a laptop and identify all its internal parts

1.3. Compare and contrast laptop and desktop components

Both desktop and laptop computers can perform most of the same functions. However, these two kinds of computers are built very differently and the parts are not interchangeable. Few components can be shared between desktops and laptops like the expansion devices that are attached to laptop devices with USB ports, parallel ports, and PC Cards.



Figure 1.6: Laptop vs Desktop

Laptop components are proprietary. So you may not be able to use components made by one laptop manufacturer to repair a laptop made by another manufacturer. Therefore, that technicians may have to obtain certification for each laptop manufacturer they support. The following table summarizes the specifications between a laptop and desktop computers.

Feature/Component	Laptop	Desktop
Motherboard form factor	Specialized components (specific form factors)	Standardized components (Universal form factors)
Processor	Processors use less power	Processors use great electricity quantity
	Processors create less heat	Processors create great heat quantity
	Processors don't require large cooling devices	Processors require large cooling devices
Power management	laptop takes electricity from the battery	Desktops remain plugged into a power source
Expansion capability	Expansion devices are attached to laptop devices with USB ports, parallel ports, and PC Cards.	A desktop attaches these devices with USB ports and parallel ports.
RAM slot type	SODIMM	DIMM
Physical Size of Hard Disk	2.5-inch drives	3.5-inch drives

Table 1.1. comparison of laptop and desktop

Expansion devices are attached to laptops and desktops differently. Laptops have limited space so the expansion bays on laptops are designed to allow different types of drives to fit into the same bay. Drives are hot-swappable and are inserted or removed as needed.

As both desktop and laptop computers are electronic devices, they manage power. The optimization of power management can be configured differently through the Control Panel feature of the running operating system. This configuration of power management can also mark the difference between a laptop and a desktop.

Application Activity 1.3

Go through the steps for power option Edit plan. To do this in Windows 10 go through these windows: "Control panel"window, "Hardware and Sound"window, "Power Options" window, Create a Power Plan and Edit Plan Setting. Then:

1. Set your laptop so that it can turn off the display:
    i. After 3 minutes when powered by battery
    ii. After 7 minutes when plugged in
3. Discuss different reasons why there is no need of large cooling devices in a computer lab containing laptops.
4. Discuss the similarities and differences between a laptop and desktop expansion components
5. Open a laptop and a desktop and identify the difference between their motherboards

1.4. SMARTPHONES, PDAs, TABLETS

Learning Activity 1.4.

All companies deal with large amount of data. Using different portable devices, users can collect and access data from the outside of the organization.

1. Identify the portable devices other than laptops that are used in your school to allow teacher and student to access materials on Internet.
2. Discuss the common uses and features of those identified devices in above question

The Smartphones, PDAs, tablets are new types of small personal computing devices that have been introduced in the mid-1990s and these are referred to as handheld computers. This type of computer is named handheld computer because it can fit in one hand while you can operate it with the other hand. Because of its reduced size, the screen of handheld computer is quite small.

1.4.1. Common uses and standard services of Smartphones, PDAs and tablets

Like the laptops, these computers are used by mobile employees whose jobs require them to move from place to place.

They are used in different fields like the following:

- Communication (example: - when you send and receive an email)
  - Voice over Internet Protocol (VoIP) – voice/video calls.
  - Instant text message ex: chatting using WhatsApp
  - Streaming music/video content
  - Communication via social networking applications)
- Entertainment (example: when you play games and watch movies while traveling)
- Business (Electronic commerce uses these computers for example in marketing and selling products or goods and services.)
- Education field (Smartphones, PDAs and tablets are teaching and learning tools. They are used in performing calculations, note taking etc.)
- Smartphones, PDAs and tablets have storage which can store information like contact information (names, addresses, phone numbers, e-mail addresses)
- Smartphones, PDAs and tablets can remind you of appointments (clock, alarm functions)

1.4.2. Common features of Smartphones, PDAs and tablets

Smartphones, PDAs, tablets have operating systems which allows them to run multiple applications. Their Operating systems have features like contacts, email, calendar, appointments, tasks, games, media etc.

Examples of mobile operating systems:

a. Apple's iOS
b. Android OS
c. Windows Phone OS
d. BlackBerry OS



Figure 1.7: Apple's iOS, Android OS, Windows Phone OS and BlackBerry OS Icons

Apart from the operating system, tablets have in common a wi-fi, capability to have their memories expanded, a high resolution display with touch technology and a QUERT keyboard

Application Activity 1.4.

Using a smartphone or a tablet or IPad do the following:

1. Download and install one of these applications from Play Store: WhatsApp, Google Chrome and Facebook.
2. Send an email message to one of the classmates and ask him/her to reply.
3. Search the application called "Polaris Office" in Play Store install it and state its function?

1.4.3. Standards Service of Smartphone, PDA and Tablet

Some of the services that can be found in smartphones, PDA, and tablets include;

- Appointment calendar
- MP3 player
- Web browser
- E-mail access, in addition to text messaging
- Mini-keyboards or onscreen keyboards
- Bluetooth
- Character recognition (allowing for handwritten input)
- Synchronization of information with desktop or laptop computers
- Voice recording
- Digital camera
- Video recording
- Location tracking through GPS(Global Positioning System)
- Wi-Fi
- Microsoft Office (MS) compatible applications (Native with Pocket PC operating systems; Palm operating systems may require third-party software)

1.5. LAPTOP AND PORTABLE DEVICES CLEANING PROCEDURES

Learning Activity 1.5

Observe the figure below and answer the followed questions



1. Discuss what would happen if a drink was spilled into the keyboard of a laptop.
2. How can you prevent this laptop to be damaged?
3. What can you do if the laptop and water are in contact?
4. What can you consider while choosing cleaning materials and products?
5. Describe different laptop parts which can be cleaned.
6. How can you keep waste products after cleaning a laptop?

As laptops are mobile, they are used in different types of environments. Some environments can be hazardous to a laptop. Even eating or drinking around a laptop creates a potentially hazardous condition.

It is important to keep a laptop clean and to ensure that it is being used in the most optimal environment possible. Proper routine cleaning is the easiest, least expensive way to protect and to extend the life of a laptop.

It is very important to use the right products, cleaning materials and procedures when cleaning a laptop. Always read all warning labels on the cleaning products.

Before starting to clean any part of a laptop, turn it off, disconnect all attached devices, disconnect it from the electrical outlet and remove its battery. To remove the battery move the battery lock to the unlocked position, hold the release lever in the unlock position and remove the battery.

To clean laptop keyboards blow compressed air between the keys then slide a soft cloth over the laptop keyboard in order to remove the dirt. The cleaning can be done using compressed air whereby this air is blown into the laptop through the vents. Use tweezers to remove any debris.

To clean LCD screens slide a soft cloth slightly moistered with water or LCD cleaner over the laptop display. A special attention should be taken on using products specifically designed for cleaning LCD displays.

To clean touch pads slide a soft cloth over surface of the touch pad gently with an approved cleaner. Never use a wet cloth and use a soft cloth with only approved cleaning solution.

Application Activity1.5

1. Clean a keyboard and a screen of a laptopin the computer lab.
2. Discuss different Laptop Keyboard, LCD screen and touchpad Cleaning materials, products and Procedures.

END OF UNIT ASSESSMENT ACTIVITY

1. Discuss the advantages and disadvantages of each type of computer below:

Laptop computer

Tablet

Smartphone

2. Describe what is used for standalone and networked laptop computer.
3. Compare smartphone features and cell phone features
4. Is a Laptop hardware maintenance difficult comparing to a desktop computer maintenance? Explain.
5. Give a clear difference between VGA and HDMI connectors.
6. Explain how you can take care your laptop.
7. Exchange a track of music from one device to another. What do you use? Are there other

UNIT 2
COMPLEX DATA STRUCTURE AND ALGORITHM

UNIT2: COMPLEX DATA STRUCTURE AND ALGORITHM

INTRODUCTORY ACTIVITY

Read the following scenario and after answer questions:

In a bank, a Teller has the role of serving the clients. He/she has to organize the banknotes of same values together and coins of same values together that he/she has to serve to clients. When a client arrives in the bank, he/she chooses a line to join in the front of a Teller and the line is formed according to the arrival of clients.

Mr. KARENZI wants to pay 78450 Rwf as his school fees to the bank and he must organize banknotes and coins before deposit money to facilitate the teller to not mix the banknotes and coins



Answer the following questions:

1. Observe the above figure and describe what you see;
2. Where a new arrived client will line up?
3. Using the above figure who is going to be served first by the teller 1 or 2;
4. Discuss how KARENZI is going to organize bank notes and coins before depositing;
5. Discuss how the teller is going to keep money deposited by Karenzi

2.1. Introduction to Data Structures

Data Structure is a way of collecting and organizing data in such a way that we can perform operations on these data in an effective way. Data Structures is about rendering data elements in terms of some relationship, for better organization and storage.

Example: Library is composed of elements (books) to access a particular book requires knowledge of the arrangement of the books.

2.1.1. Basic types of Data Structures

Anything that can store data can be called as a data structure, hence Integer, Float, Boolean, Char etc., all are data structures. They are known as **Primitive Data Structures**. And we also have some complex Data Structures, which are used to store large and connected data and are called **non primitive** data structure or **Abstract Data Structure**.

Example: Array, Linked List, Tree, Graph, Stack, Queue etc.

All these data structures allow us to perform different operations on data. We select these data structures based on which type of operation is required.

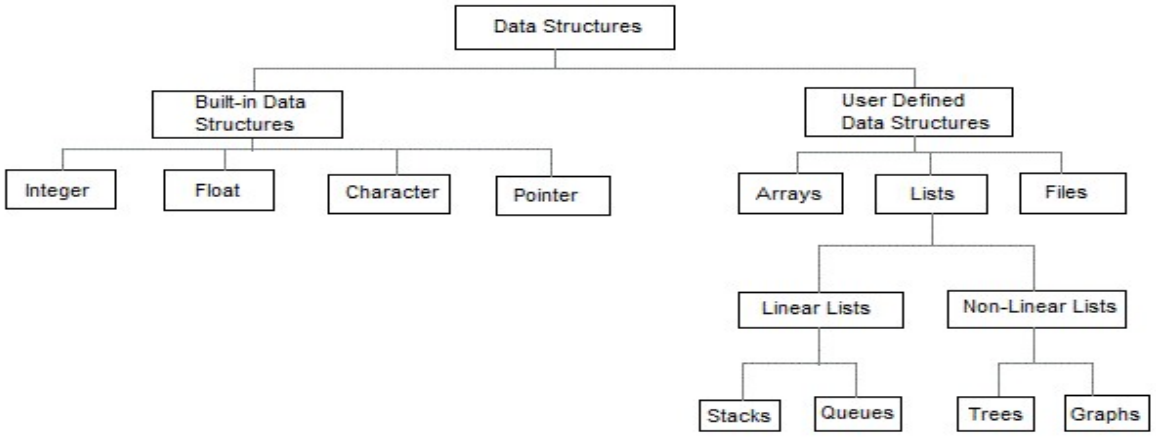


Figure 2.1. Abstract data structure

2.1.2. Importance of data structures

The data structures have the following importance for programming:

- Data structures study how data are stored in a computer so that operations can be implemented efficiently
- Data structures are especially important when there is a large amount of information to deal with.
- Data structures are conceptual and concrete ways to organize data for efficient storage and manipulation

2.1.3. Data Structure Operations

Beside the simple operations performed on data (unary operators, binary operators and precedence operators) seen in Senior Four, the data appearing in our data structures are processed by means of other operations. The following are some of the most frequently used operations.

Traversing: Accessing each record exactly once so that certain items in the record may be processed.

- **Searching**: Finding the location of the record with a given key value or finding the locations of all records which satisfy one or more conditions
- **Inserting:** Adding a new record to the structure.
- **Deleting:** Removing a record from the structure.
- **Sorting:** Arranging the records in some logical order (e.g., in numerical order according to some NUMBER key, such as social security number or account number).
- **Merging:** Combining the records of two different sorted lists into a single sorted list.

It is important for every Computer Science student to understand the concept of *Information* and how it is organized or how it can be utilized.

If we arrange some data in appropriate sequence, then it forms a Structure and gives us a meaning. This meaning is called information. The basic unit of information in computer Science is a bit, binary digit. So we found two things in information: One is **Data** and the other is **Structure**

2.2 Array of elements in data structure

LEARNING ACTIVITY 2.1.

Study the following table

	Mathematics	Physics	Computer
Niyibizi			
Urakaza			
Shyaka			
kwizera		95	

	Mathematics	Physics	Computer
Niyibizi			
Urakaza			
Shyaka			
kwizera		95	

Answer the following questions.

1. Suggest a name for the above table?
2. How many rows and columns in the above table?
3. Identify the location of the value 95 in the above table?

Arrays hold a series of data elements, usually of the same size and data type. Individual elements are accessed by their position in the array. The position is given by an **index**, which is also called a **subscript**. The index usually uses a consecutive range of integers, (as opposed to an associative array) but the index can have any ordinal set of values

Some arrays are *multi-dimensional,* meaning they are indexed by a fixed number of integers, for example by a tuple of four integers. Generally, one- and two-dimensional arrays are the most common. Most programming languages have a built-in *array* data type.

In computer programming, a group of homogeneous elements of a specific data type is known as **an array,** one of the simplest data structures.

2.2.1. Arrays dimensions:

A dimension of array is a direction in which you can vary the specification of an array›s elements. An array can be of one or many dimensions.

a. One-dimensional array.

Example: a [3]

It is an array called a with 3 elements. The elements of the array are logically represented by the name of the array with in brackets its index or positions. For the one dimensional array a with 3 elements, elements are written like a[0], a[1] and a[2]. The indexes start always by 0. Graphically, it is

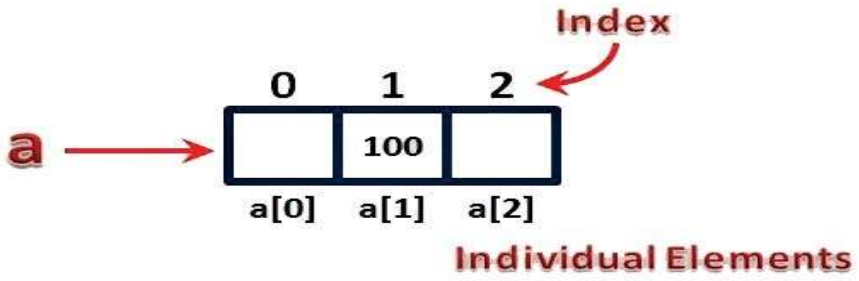


Figure 2.2.: One-dimensional array

b. Multi-dimensional arrays

Example:

Two dimensional array ex: a[2][7]as Integer

Three dimensional array ex: a[3][2][5]as Integer

2.2.2. Two dimensional array

A Two dimensional Array is a collection of a fixed number of elements (components) of the same type arranged in two dimensions.

The two dimensional array is also called a matrix or a table. The intersection of a column and a row is called a cell. The numbering of rows and columns starts by 0

Example:
The array a[3][4] is an array of 3 rows and 4 columns. In matrix representation, it looks like the following:

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Figure 2.3.: Two-dimensional array

a[i][j] means that it is an element located at the intersection of row i and column j.

Each element is identified by its row and column

2.2.3. Two dimensional array declaration

The syntax for declaring a two-dimensional array is:

SET Array name=Array [row size][column size] of Data type

Where row size and column size are expressions yielding positive integer values, the two expressions row size and column size specify the number of rows and the number of columns, respectively, in the array.

Examples:

1. SET a=Array[2][7] of Integer
2. SET b=Array[4][3] of Integer

Notice that the number of elements **=row size*column size**

The examples above defines respectively arrays named a and b with 14 (2*7) elements and 12 (4*3) elements. The expression marks [0] [0] will access the first element of the matrix marks and marks [1] [6] will access the last row and last column of array a.

If the array marks if filled with the elements 80, 75, 76, 75, 55, 70, 55, 70, 65, 85, 35 and 59, we get the following table.

2.2.4. Two dimensional array initialization

When a two-dimension array has been declared, it needs to be assigned values for its elements. This action of assigning values to array elements is called initialization. Two-Dimensional array is initialized by specifying bracketed values for each row. For the case of array of integers, the default values of each element is 0 while for others the default values for other types is not known.

For the case of an array of 3 rows and 4 columns, if the name is A and the type of elements is integer, the initialization is:

int A[3][4]={{A[0][0], A[0][1], A[0][2], A[0][3]}, /* initializers for row indexed by 0 */

{A[1][0], A[1][1], A[1][2], A[1][3]}, /* initializers for row indexed by 1*/

{A[2][0], A[2][1], A[2][2], A[2][3]} /* initializers for row indexed by 2 */

};

The nested braces, which indicate the intended row, are optional. The following initialization is equivalent to the previous initialization.

Int A[3][4]={A[0][0], A[0][1], A[0][2], A[0][3], A[1][0], A[1][1], A[1][2], A[1][3], A[2][0], A[2][1], A[2][2], A[2][3]};

An example of marks for 3 students in 4 courses. The array will look like this:

	col 0	col 1	col 2	col 3
row 0	18	16	14	19
row 1	10	20	12	15
row 2	15	17	18	16

Table 2.1.: Array initialization

If the table is called marks, marks [1][3]=15; where the row position is 1 and the column position is 3.

The expression marks [0] [0] will access the first element of the matrix marks and marks [2] [3] will access the last row and last column.

Syntax to initializing two-dimensional arrays

The two dimensional array marks which will hold 12 elements can be initialized like:

BEGIN
SET marks=Array[3] [4] of Integer
marks[0][0]=18

marks[0][1]=16
marks[0][2]=14
marks[0][3]=19
marks[1][0]=10
marks[1][1]=20
marks[1][2]=12
marks[1][3]=15
marks[2][0]=15
marks[2][1]=17
marks[2][2]=18
marks[2][3]=16
END

APPLICATION ACTIVITY 2.1.

1. Four secondary schools in RUHANGO District are displaying the number of students per year that they registered in 2015, 2016, 2017 and 2018.
   a. How many rows and columns will be in the table that contains the displayed data?
   b. Declare an array called population to contain the needed data.
   c. Initialize the array population with numbers of your choice.
2. Given below the case of array a:

a[0][0]	1	a[1][0]	54
a[0][1]	2	a[1][1]	23
a[0][2]	3	a[1][2]	11
a[0][3]	13	a[1][3]	45
a[0][4]	17	a[1][4]	31
a[0][5]	34	a[1][5]	65
a[0][6]	19	a[1][6]	26

   a. Give the size of the above array;
   b. Declare array a;
   c. Initialize the above array.

2.2.5. Accessing two dimensional array elements

LEARNING ACTIVITY 2.2.

What do the following statements mean:

WRITE marks[1][2] ;

What are the meanings of the above statements

- **Reading (storing) two dimensional array elements**

  There exist two ways of accessing the elements of an array. The elements' values of the array can come from the input devices or from a given file so that these elements are written into the array. The function used is READ or other versions of it, depending on the programming languages used for programs writing.

  For example READ marks[1][2] stores a value in second row, third column location of the array marks.

  To store multiple values into array you must use a FOR loop.

Syntax:

FOR i=initial value TO upperlimit DO
FOR j=initial value TO upperlimit DO
READ array name[i][j]
i=i+1
j=j+1
END FOR
END FOR

Where i refers to the row position and j refers to the column position.

Initial value=lower limit

Example: Write an algorithm of a program that allows the user to enter (store) two dimensional array elements

BEGIN
SET marks=Array [3] [8] of Integer
Use variable i as integer
Use variable j as integer
FOR i=0 TO 2 DO
FOR j=0 TO 7 DO
READ marks[i] [j]
i=i+1
j=j+1

END FOR
END FOR
END

Writing (displaying) two dimensional array elements

To access two dimensional array elements' values, we can also take the elements' values of an array residing in the memory of the computer and write them in any output device or a given file.

To write (display) elements from an array in an output device or other file, use a WRITE function or other versions of it depending on the programming languages used for programs writing together with the array name and location (index) of the element. In that case the elements of a two dimensional array can be displayed by the following syntax:

WRITE Array name [i][j]

Where i refers to the row position and j refers to the column position.

To display a single value of an array you must provide the array name and index to write operation example:

Example for writing on screen a two dimensional array element

WRITE a[1][2], i=1 and j=2).

Example: Write an algorithm of a program that displays stored two dimensional array marks' elements

BEGIN
SET a=Array[3][8] of integer
Use variable i As integer
Use variable j As integer
WRITE "stored two dimensional array elements are:"
FOR i=0 to 2DO
FOR j=0 to 7DO
WRITE a[i][j]
i=i+1
j= j+1
END FOR
END FOR
END

Algorithm for reading and printing values in arrays

Example: Write an algorithm that allows the user to enter(store) two dimensional array elements and displays those stored elements.

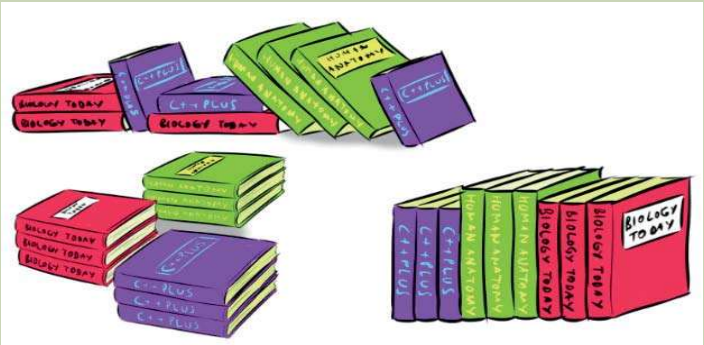
```
BEGIN
SET Scores=Array[3][8] of Integer
Use variable i  As integer
Use variable j As integer
WRITE "Enter  two dimensional array
  elements:"
FOR i=0 TO 2 DO
 FOR j=0 TO 7 DO
READ Scores[i][j]
i=i+1
j=j+1
END FOR
END FOR
WRITE "stored  two dimensional array
  elements are:"
FOR  i=0 to 2 DO
FOR j=0 to 7 DO
WRITE  a[i][j]
i=i+1
j= j+1
END FOR
END FOR
END
```

APPLICATION ACTIVITY 2.2.

1. Write an algorithm to do the following operations on a two dimensional array A of size m x n.
   - Declare the array A;
   - Read the element 'values into matrix of size m x n
   - Output the elements' values of matrix of size m x n on the screen;
   - Calculate the sum of all elements' values of matrix of size m x n;
   - Write a programme in C++ Language where you apply the above questions
2. Write an algorithm that allows a user to input data through input device and display the smallest element' s value on the screen.
3. Write a programme in C++ language that implements the above algorithm.
4. Consider a 2 by 3 array T
   a. Write a declaration of T
   b. How many rows does T have?
   c. How many columns does T have?
   d. How many elements does T have?
   e. Write name of elements in second row
   f. Write the name of element in third column
   g. Write a single statement that sets the elements of T in row 1and column 2 to zero
   h. Write a statement that inputs the values of T from keyboard
   i. Write a statement that displays the elements of first row of T

2.3. ABSTRACT DATA STRUCTURE

LEARNING ACTIVITY 2.3



1. Observe and analyze the above figure
2. Explain different procedures that you can use to arrange the above books.

2.3.1. Definition of Abstract Data type in data structure.

Abstract Data structure is a mathematical model of a data structure that specifies the type of data stored, the operations supported on them and the types of parameters of the operations with no implementation. Each program language has its own way to implement abstract data type.

2.3.2 Importance of abstract data type in data structures

- Data structures study how data are stored in a computer so that operations can be implemented efficiently;
- Abstract Data structures are especially important when you have a large amount of information;
- Conceptual and concrete ways to organize data for efficient storage and manipulation

2.3.3. Abstract Data type Operations

The data appearing in our data structures are processed by means of certain operations. Following are some of the most frequently used operations.

- **Searching:** Finding the location of the record with a given key value or finding the locations of all records which satisfy one or more conditions;
- **Sorting:** Arranging the records in some logical order .

2.4. List

2.4.1. Introduction

A list is a collection of data in which each element of the list contains the location of the next element.

In a list, each element contains two parts: data and link to the next element. The data parts of the list hold the value information and data to be processed. The link part of the list is used to chain the data together and contains a pointer (an address) that identifies the next element in the list.

In the list you can do the following operations:

- LIST: Creates an empty list;
- INSERT: inserts an element in the list;
- DELETE: deletes an element from the list;
- RETRIEVE: retrieves an element from the list;
- TRAVERSE: traverses the list sequentially;
- EMPTY: checks the status of the list.

Example of a list:

A bank has 10 branches created one after another at different dates. The Bank Inspector who wants to visit all of them can follow the order of their creation. He/she will be informed only about the name and location of the first branch. When he reaches that branch, the Branch Manager will tell him/her the address of the next branch, and so one.

After visiting all branches, the Bank Inspector will realize that he/she has a list of the bank branches.

Graphically, it will be represented like a list where each branch is considered as a node containing the names of the branch and the address of the next branch.

Difference between Arrays and Lists

A list is an ordered set of data. It is often used to store objects that are to be processed sequentially.

An array is an indexed set of variables, such as dancer [1], dancer [2], dancer [3] It is like a set of boxes that hold things. A list is a set of items. An array is a set of variables that each store an item.

In a list, the missing spot is filled in when something is deleted. While in an array, an empty variable is left behind when something is deleted.Simply a list is a sequence of data, and linked list is a sequence of data linked with each other.

2.4.2 Linked list

This is a dynamic table/ data structure used to hold a sequence. Its characteristics are the following:

- Items forming a sequence are not necessarily held in contiguous data location or in the order in which they occur in the sequence.
- Each item in the list is called a node and contains an information field and a new address field called a link or pointer field.
- Information field holds actual data for list item; link field contains the address of the next item.
- The link field in the last item indicates that there are not further items. E.g. has a value of 0.
- Associated with the list is a pointer variable which points to the first node in the list.
- We use linked list in linking files, for example, in a hard disk.
- It can grow or shrink in size during execution of a program.
- It can be made just as long as required
- It does not waste memory space.

Linked list is classified into three types as shown below:

- Single linked lists
- Double linked lists
- Circular linked list

1. Single linked lists

Single linked list is a sequence of elements in which every element has link to its next element in the sequence. In any single linked list, the individual element is called as "Node". Every "Node" contains two fields, data and next. The data field is used to store actual value of that node and next field is used to store the address of the next node in the sequence



Figure 2.6: Node representation

In a single linked list, the address of the first node is always stored in a reference node known as "front" (Sometimes it is also known as "head").Always next part (reference

part) of the last node must be NULL

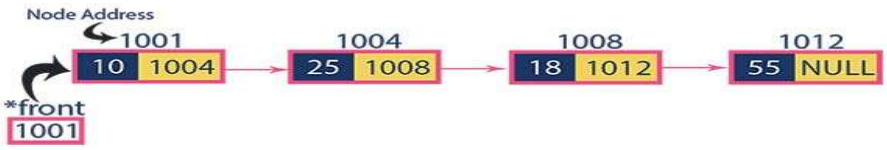


Figure 2.7: Single Linked List representation

Algorithm in Single Linked List

Creating and read an element in a NODE

BEGIN
struct node{
int info
struct node next
} head, current, temp
head=null
temp=null
current=null
Begin sub

Step 1 Read the element into x
Step 2 Create a temp node in the memory
temp =(struct node)sizeof (node)
Step 3 Assign the values in temp node as follows
temp -> info =x
temp ->next=null
End sub
END

Inserting at Specific location in the list (After a Node)

We can use the following steps to insert a new node after a node in the single linked list

- **Step 1:** Create a **newNode** with given value.
- **Step 2:** Check whether list is **Empty** (**head** == **NULL**)
- **Step 3:** If it is **Empty** then, set **newNode => next** = **NULL** and **head** = **newNode**.
- **Step 4:** If it is **Not Empty** then, define a node pointer **temp** and initialize with **head**.
- **Step 5:** Keep moving the **temp** to its next node until it reaches to the node

after which we want to insert the newNode (until **temp1 => data** is equal to **location**, here location is the node value after which we want to insert the newNode).

- **Step 6:** Every time check whether **temp** is reached to last node or not. If it is reached to last node then display '**Given node is not found in the list!!! Insertion not possible!!!'** and termin ate the function. Otherwise move the **temp** to next node.
- **Step 7:** Finally, Set '**newNode => next** = **temp => next**' and '**temp => next** = **newNode**'

Deleting a Specific Node from the list

We can use the following steps to delete a specific node from a single linked list

- **Step 1:** Check whether list is Empty (head == NULL)
- **Step 2:** If it is Empty then, display 'List is Empty!!! Deletion is not possible' and terminate the function.
- **Step 3:** If it is Not Empty then, define two Node pointers 'temp1' and 'temp2' and initialize 'temp1' with head.
- **Step 4:** Keep moving the temp1 until it reaches to the exact node to be deleted or to the last node. And every time set 'temp2 = temp1' before moving the 'temp1' to its next node.
- **Step 5:** If it is reached to the last node then display 'Given node not found in the list! Deletion not possible!!!' And terminate the function.
- **Step 6:** If it is reached to the exact node which we want to delete, then check whether list is having only one node or not
- **Step 7:** If list has only one node and that is the node to be deleted, then set head = NULL and delete temp1 (free (temp1)).
- **Step 8:** If list contains multiple nodes, then check whether temp1 is the first node in the list (temp1 == head).
- **Step 9:** If temp1 is the first node then move the head to the next node (head = head => next) and delete temp1.
- **Step 10:** If temp1 is not first node then check whether it is last node in the list (temp1=> next == NULL).
- **Step 11:** If temp1 is last node then set temp2 => next = NULL and delete temp1 (free (temp1)).
- **Step 12:** If temp1 is not first node and not last node then set temp2 => next = temp1=> next and delete temp1 (free (temp1)).

Displaying a Single Linked List

We can use the following steps to display the elements of a single linked list

- **Step 1:** Check whether list is Empty (head == NULL)
- **Step 2:** If it is Empty then, display 'List is Empty!!!' and terminate the function.
- **Step 3:** If it is Not Empty then, define a Node pointer 'temp' and initialize with head.
- **Step 4:** Keep displaying temp=> data with an arrow (->) until temp reaches to the last node
- **Step 5:** Finally display temp => data with arrow pointing to NULL (temp => data -> NULL).

2. Double linked lists

Double linked list is a sequence of elements in which every element has links to its previous element and next element in the sequence. So, we can traverse forward by using next field and can traverse backward by using previous field. Every node in a double linked list contains three fields and they are shown in the following figure

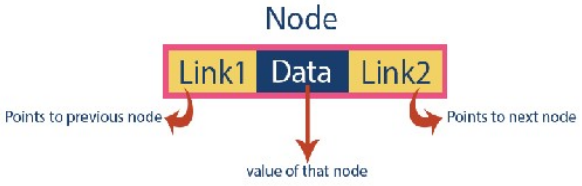


Figure 2.8: Representation of double linked list

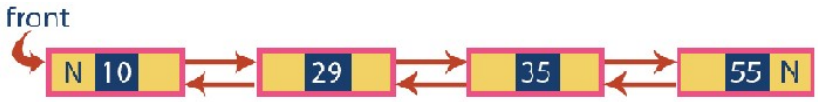


Figure 2.9: Double linked list representation

Algorithm in Double Linked List

Inserting at Specific location in the list (After a Node)

We can use the following steps to insert a new node after a node in the double linked list

- **Step 1:** Create a newNode with given value.
- **Step 2:** Check whether list is Empty (head == NULL)
- **Step 3:** If it is Empty then, assign NULL to newNode => previous&newNode => next and newNode to head.
- **Step 4:** If it is not Empty then, define two node pointers temp1&temp2 and

initialize temp1 with head.

- **Step 5:** Keep moving the temp1 to its next node until it reaches to the node after which we want to insert the newNode (until temp1 => data is equal to location, here location is the node value after which we want to insert the newNode).
- **Step 6:** Every time check whether temp1 is reached to the last node. If it is reached to the last node then display 'Given node is not found in the list!!! Insertion not possible!!!' and terminate the function. Otherwise move the temp1 to next node.
- **Step 7:** Assign temp1 => next to temp2, newNode to temp1=> next, temp1 to newNode => previous, temp2 to newNode => next and newNode to temp2 => previous.

Deleting a Specific Node from the list

We can use the following steps to delete a specific node from the double linked list

- **Step 1:** Check whether list is Empty (head == NULL)
- **Step 2:** If it is Empty then, display 'List is Empty!!! Deletion is not possible' and terminate the function.
- **Step 3:** If it is not Empty, then define a Node pointer 'temp' and initialize with head.
- **Step 4:** Keep moving the temp until it reaches to the exact node to be deleted or to the last node.
- **Step 5:** If it is reached to the last node, then display 'Given node not found in the list! Deletion not possible!!!' and terminate the fuction.
- **Step 6:** If it is reached to the exact node which we want to delete, then check whether list is having only one node or not
- **Step 7:** If list has only one node and that is the node which is to be deleted then set head to NULL and delete temp (free(temp)).
- **Step 8:** If list contains multiple nodes, then check whether temp is the first node in the list (temp == head).
- **Step 9:** If temp is the first node, then move the head to the next node (head = head => next), set head of previous to NULL (head => previous = NULL) and delete temp.
- **Step 10:** If temp is not the first node, then check whether it is the last node in the list (temp => next == NULL).
- **Step 11:** If temp is the last node then set temp of previous of next to NULL (temp => previous => next = NULL) and delete temp (free(temp)).

- **Step 12:** If temp is not the first node and not the last node, then set temp of previous of next to temp of next (temp => previous => next = temp => next), temp of next of previous to temp of previous (temp => next => previous = temp => previous) and delete temp (free(temp)).

Displaying a Double Linked List

We can use the following steps to display the elements of a double linked list

- **Step 1:** Check whether list is **Empty** (**head** == **NULL**)
- **Step 2:** If it is **Empty**, then display **'List is Empty!!!'** and terminate the function.
- **Step 3:** If it is not Empty, then define a Node pointer **'temp'** and initialize with **head**.
- **Step 4:** Display **'NULL <= '**.
- **Step 5:** Keep displaying **temp → data** with an arrow (⇔) until **temp** reaches to the last node
- **Step 6:** Finally, display **temp → data** with arrow pointing to **NULL** (**temp → data ---> NULL**).

3. Circular Linked List

In circular linked list, every node points to its next node in the sequence but the last node points to the first node in the list

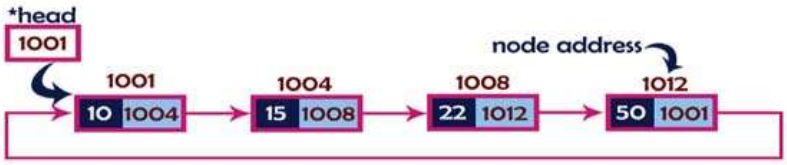


Figure 2.10: Circular Linked List representation

Algorithm in Circular Linked List

Inserting at Specific location in the list (After a Node)

We can use the following steps to insert a new node after a node in the circular linked list...

- **Step 1:** Create a **newNode** with given value.
- **Step 2:** Check whether list is **Empty** (**head** == **NULL**)
- **Step 3:** If it is **Empty** then, set **head** = **newNode** and **newNode** → **next** = **head**.
- **Step 4:** If it is **Not Empty** then, define a node pointer **temp** and initialize with **head**.

- **Step 5:** Keep moving the **temp** to its next node until it reaches to the node after which we want to insert the newNode (until **temp1 → data** is equal to **location**, here location is the node value after which we want to insert the newNode).
- **Step 6:** Every time check whether **temp** is reached to the last node or not. If it is reached to last node then display **'Given node is not found in the list!!! Insertion not possible!!!'** and terminate the function. Otherwise move the **temp** to next node.
- **Step 7:** If **temp** is reached to the exact node after which we want to insert the newNode then check whether it is last node (temp → next == head).
- **Step 8:** If **temp** is last node then set **temp → next** = **newNode** and **newNode → next** = **head**.
- **Step 8:** If **temp** is not last node then set **newNode → next** = **temp → next** and **temp → next** = **newNode**.

Deleting a Specific Node from the list

We can use the following steps to delete a specific node from the circular linked list...

- **Step 1:** Check whether list is **Empty** (**head** == **NULL**)
- **Step 2:** If it is **Empty** then, display **'List is Empty!!! Deletion is not possible'** and terminate the function.
- **Step 3:** If it is **Not Empty** then, define two Node pointers **'temp1'** and '**temp2'** and initialize '**temp1**' with **head**.
- **Step 4:** Keep moving the **temp1** until it reaches to the exact node to be deleted or to the last node. And every time set '**temp2 = temp1**' before moving the '**temp1**' to its next node.
- **Step 5:** If it is reached to the last node then display **'Given node not found in the list! Deletion not possible!!!'**. And terminate the function.
- **Step 6:** If it is reached to the exact node which we want to delete, then check whether list is having only one node (**temp1 → next** == **head**)
- **Step 7:** If list has only one node and that is the node to be deleted then set **head** = **NULL** and delete **temp1** (**free(temp1)**).
- **Step 8:** If list contains multiple nodes then check whether **temp1** is the first node in the list (**temp1 == head**).
- **Step 9:** If **temp1** is the first node then set **temp2** = **head** and keep moving **temp2** to its next node until **temp2** reaches to the last node. Then set **head = head → next**, **temp2 → nex**t = **head** and delete **temp1**.
- **Step 10:** If **temp1** is not first node then check whether it is last node in the list

(**temp1 → next == head**).

- **Step 11:** If **temp1** is last node then set **temp2 → next** = **head** and delete **temp1** (**free (temp1)**).
- **Step 12:** If **temp1** is not first node and not last node then set **temp2 → next** = **temp1 → next** and delete **temp1** (**free (temp1)**).

Displaying a circular Linked List

We can use the following steps to display the elements of a circular linked list

- **Step 1:** Check whether list is **Empty** (**head** == **NULL**)
- **Step 2:** If it is **Empty**, then display **'List is Empty!!!'** and terminate the function.
- **Step 3:** If it is **Not Empty** then, define a Node pointer **'temp'** and initialize with **head**.
- **Step 4:** Keep displaying **temp → data** with an arrow (**--->**) until **temp** reaches to the last node
- **Step 5:** Finally display **temp → data** with arrow pointing to **head → data**.

2.5. Queue

A queue is a linear list in which data can only be inserted at one end, called the REAR, and deleted from the other end, called the FRONT. These restrictions ensure that the data is processed through the queue in the order in which it is received. In an other words, a queue is a structure in which whatever goes fist comes out first (**first in, first out(FIFO)** structure).

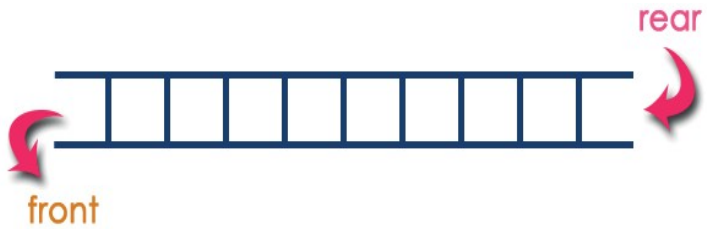


Figure 2.11: Queue representation

2.5.1 Types of queues

There exist two types of queues:

- Linear queue
- Circular queue

a. Linear Queue

Queue data structure is a linear data structure in which the operations

are performed based on FIFO principle. **Queue Operations using Array** before we implement actual operations, first follow the below steps to **create an empty queue**.

- **Step 1:** Include all the **header files** which are used in the program and define a constant **'SIZE'** with specific value.
- **Step 2:** Declare all the **user defined functions** which are used in queue implementation.
- **Step 3:** Create a one dimensional array with above defined SIZE (**int queue[SIZE]**)
- **Step 4:** Define two integer variables **'front'** and '**rear**' and initialize both with **'-1'**. (**int front = -1, rear = -1**)
- **Step 5:** Then implement main method by displaying menu of operations list and make suitable function calls to perform operation selected by the user on queue.

enQueue(value) - Inserting value into the queue

- Step 1: Check whether queue is FULL. (rear == SIZE-1)
- Step 2: If it is FULL, then display "Queue is FULL!!! Insertion is not possible!!!"" and terminate the function.
- Step 3: If it is NOT FULL, then increment rear value by one (rear++) and set queue [rear] = value.

deQueue() - Deleting a value from the Queue

- Step 1: Check whether queue is EMPTY. (front == rear)
- Step 2: If it is EMPTY, then display "Queue is EMPTY!!! Deletion is not possible!!!" and terminate the function.
- Step 3: If it is NOT EMPTY, then increment the front value by one (front ++). Then display queue[front] as deleted element. Then check whether both front and rear are equal (front == rear), if it TRUE, then set both front and rear to '-1' (front = rear = -1).

display() - Displays the elements of a Queue

We can use the following steps to display the elements of a queue

- **Step 1:** Check whether **queue** is **EMPTY**. (**front == rear**)
- **Step 2:** If it is **EMPTY**, then display **"Queue is EMPTY!!!"** and terminate the function.
- **Step 3:** If it is **NOT EMPTY**, then define an integer variable '**I'** and set '**i** =

front+1'.

- **Step 3:** Display '**queue[i]'** value and increment '**I**' value by one (**i++**). Repeat the same until '**I**' value is equal to **rear** (**i**<= **rear**)

b. Circular Queue

Circular Queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle and the last position is connected back to the first position to make a circle.

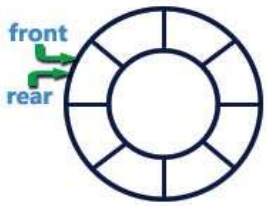


Figure 2.12: Circular Queue representation

Circular Queue Operation

To implement a circular queue data structure using array, we first **create it**.

- **Step 1:** Include all the **header files** which are used in the program and define a constant **'SIZE'** with specific value.
- **Step 2:** Declare all **user defined functions** used in circular queue implementation.
- **Step 3:** Create a one dimensional array with above defined SIZE (**int cQueue[SIZE]**)
- **Step 4:** Define two integer variables **'front'** and '**rear**' and initialize both with '**-1'**. (**int front = -1, rear = -1**)
- **Step 5:** Implement main method by displaying menu of operations list and make suitable function calls to perform operation selected by the user on circular queue.

enQueue(value) - Inserting value into the Circular Queue

- **Step 1:** Check whether **queue** is **FULL**. (**(rear == SIZE-1 && front == 0) || (front == rear+1)**)
- **Step 2:** If it is **FULL**, then display **"Queue is FULL!!! Insertion is not possible!!!"** and terminate the function.
- **Step 3:** If it is **NOT FULL**, then check **rear == SIZE - 1 && front != 0** if it is **TRUE**, then set **rear = -1**.
- **Step 4:** Increment **rear** value by one (**rear++**), set **queue[rear]** = **value** and check ‹**front == -1**' if it is **TRUE**, then set **front = 0**.

deQueue() - Deleting a value from the Circular Queue

- **Step 1:** Check whether **queue** is **EMPTY**. (**front == -1 && rear == -1**)
- **Step 2:** If it is **EMPTY**, then display **"Queue is EMPTY!!! Deletion is not possible!!!"** and terminate the function.
- **Step 3:** If it is **NOT EMPTY**, then display **queue[front]** as deleted element and increment the **front** value by one (**front ++**). Then check whether **front == SIZE**, if it is **TRUE**, then set '**front = 0'**. Then check whether both '**front – 1'** and **rear** are equal (**front -1** == **rear**), if it **TRUE**, then set both **front** and **rear** to '**-1**' (**front** = **rear** = **-1**).

display () - Displays the elements of a Circular Queue

- **Step 1:** Check whether **queue** is **EMPTY**. (**front == -1**)
- **Step 2:** If it is **EMPTY**, then display **"Queue is EMPTY!!!"** and terminate the function.
- **Step 3:** If it is **NOT EMPTY**, then define an integer variable '**i**' and set '**i** = **front**'.
- **Step 4:** Check whether '**front <= rear**', if it is **TRUE**, then display '**queue[i]**' value and increment '**I**' value by one (**i++**). Repeat the same until '**i <= rear'** becomes **FALSE**.
- **Step 5:** If '**front <= rear**' is **FALSE**, then display '**queue[i]'** value and increment '**I**' value by one (**i++**). Repeat the same until '**i <= SIZE – 1**' becomes **FALSE**.
- **Step 6:** Set **i** to **0**.
- **Step 7:** Again display '**cQueue[i]**' value and increment **i** value by one (**i++**). Repeat the same until '**i <= rear**' becomes **FALSE**.

2.5.2 Queue operations

There are three main operations related to queues.

- Enqueue: the enqueue operation inserts an items at the rear of the queue
- Dequeue: the dequeue operation deletes the item at the front of the queue
- Display: show elements in the array

2.6 Stack

A stack is a restricted linear list in which all additions and deletions are made at one end, the top. If we insert a series of data items into a stack and then remove them, the order of the data is reversed. Data input as 5, 10, 15, 20, for example would be removed as 20, 15, 10, and 5. This reversing attribute is why stacks are known as Last in, First out (LIFO) data structures.

We use many different types of stacks in our daily lives. We often talk of a stack of coins, stack of books on a table and stack of plates in a kitchen. Any situation in which

we can only add or remove an object at the top is a stack. If we want to remove an object other than the one at the top, we must first remove all objects above it. The following charts illustrates cases of stack

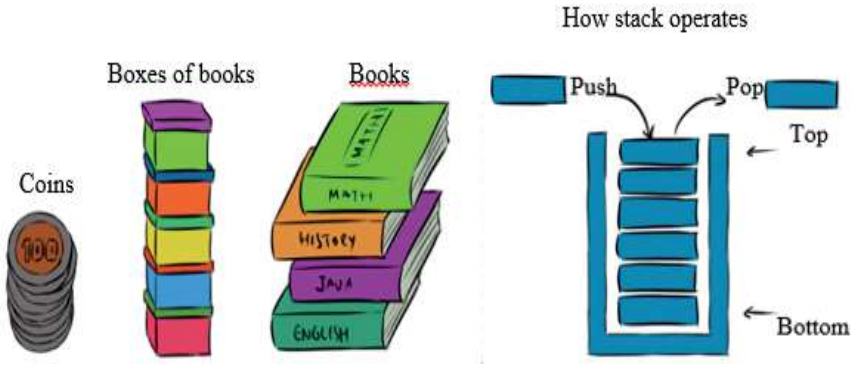


Figure 2.13: cases of stack

A Stack is a Last in First out (LIFO) dynamic table or data structure. It has the following characteristics:

- List of the same kind of elements;
- Addition and deletion of elements occur only at one end, called the top of the stack;
- Computers use stacks to implement method calls;
- Stacks are also used to convert recursive algorithms into non recursive algorithm.

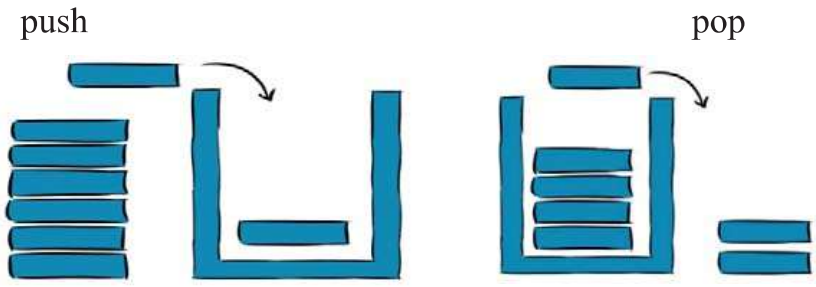


Figure 2.14: stacks' operations (Push and Pop)

2.6.1 Operations performed on stacks

The different operations performed on stacks are as follows:

- **Push**: adds an element to the stack
- **Pop**: removes an element from the stack
- **Peek**: display at top element of the stack

Stack Operations using Array

Before implementing actual operations, first follow the below steps to **create an empty stack**.

Step 1: Include all the **header files** which are used in the program and define a constant **'SIZE'** with specific value.

- **Step 2:** Declare all the **functions** used in stack implementation.
- **Step 3:** Create a one dimensional array with fixed size (**int stack[SIZE]**)
- **Step 4:** Define a integer variable '**top**' and initialize with **'-1'**. (**int top = -1**)
- **Step 5:** In main method display menu with list of operations and make suitable function calls to perform operation selected by the user on the stack.

push(value) - Inserting value into the stack

- **Step 1:** Check whether **stack** is **FULL**. (**top == SIZE-1**)
- **Step 2:** If it is **FULL**, then display **"Stack is FULL!!! Insertion is not possible!!!"** and terminate the function.
- **Step 3:** If it is **NOT FULL**, then increment **top** value by one (**top++**) and set stack [top] to value (**stack[top] = value**).

pop() - Delete a value from the Stack

- **Step 1:** Check whether **stack** is **EMPTY**. (**top == -1**)
- **Step 2:** If it is **EMPTY**, then display **"Stack is EMPTY!!! Deletion is not possible!!!'"** and terminate the function.
- **Step 3:** If it is **NOT EMPTY**, then delete **stack [top]** and decrement **top** value by one(**top-**).

Display() - Displays the elements of a Stack

- **Step 1:** Check whether **stack** is **EMPTY**. (**top == -1**)
- **Step 2:** If it is **EMPTY**, then display **"Stack is EMPTY!!!"** and terminate the function.
- **Step 3:** If it is **NOT EMPTY**, then define a variable '**I**' and initialize with top. Display **stack[i]** value and decrement **i** value by one (**i--**).
- **Step 3:** Repeat above step until **i** value becomes '0'.

2.6.2 Stack exceptions

Adding an element to a full stack and removing an element from an empty stack would generate errors or exceptions:

- **Stack overflow exception:** it occurs if you try to add an item to stack or queue which is already full.
- **Stack underflow exception:** it occurs if you try to remove an item from an empty queue or stack.

Notec 1: Stack and Queue data structures can be implement in two ways: By using array or by using linked list.

Notice 2: When stack or Queue is implemented using array, that stack or Queue can organize only a limited number of elements. When stack or Queue is implemented using linked list, that stack or Queue can organize an unlimited number of elements.

2.7 Tree

In linear data structure, data is organized in sequential order and in non-linear data structure; data is organized in random order. Tree is a very popular data structure used in wide range of applications. A tree data structure can be defined as follows.

In a tree data structure, if we have **N** number of nodes then we can have a maximum of **N-1** number of links.

Tree data structure is a collection of data (Node) which is organized in hierarchical structure and this is a recursive definition.

Example

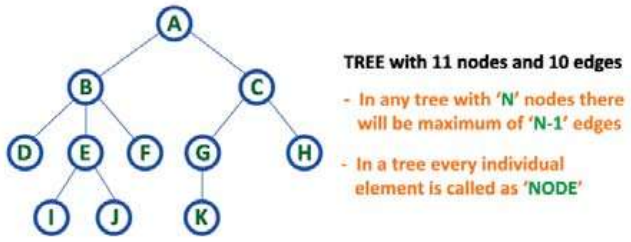


Figure 2.15.: Tree representation

2.7.1 Tree Terminology

In a tree data structure, we use the following terminology:

a. Root

In a tree data structure, the first node is called as **Root Node**. Every tree must have root node..

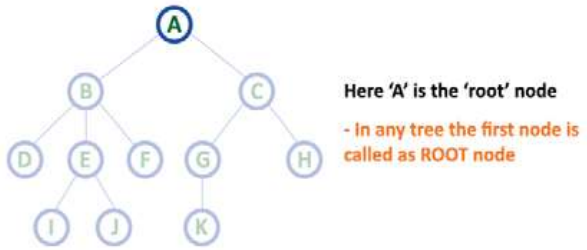


Figure 2.16: Tree Terminology representation

b. Edge

In a tree data structure, the connecting link between any two nodes is called an **Edge**. In a tree with '**N**' number of nodes there will be a maximum of '**N-1**' number of edges.

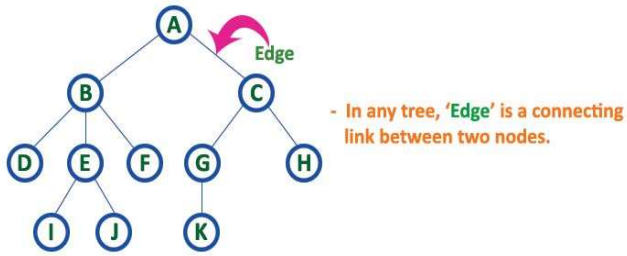


Figure 2.17: Edge representation

c. Parent

In a tree data structure, the node which is predecessor of any node is called a **PARENT NODE**. In simple words, the node which has branch from it to any other node is called as parent node.

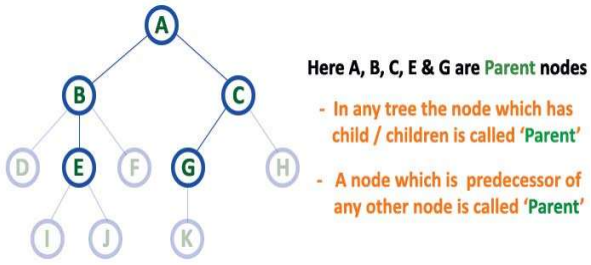


Figure 2.18.: Parent representation

d. Child

In a tree data structure, the node which is descendant of any node is called a **CHILD Node**. In a tree, any parent node can have any number of child nodes. In a tree, all the nodes except root are child nodes.

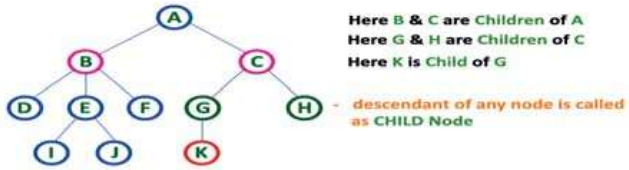


Figure 2.19.: Child representation

e. Siblings

In a tree data structure, nodes which belong to same Parent are called **SIBLINGS**.

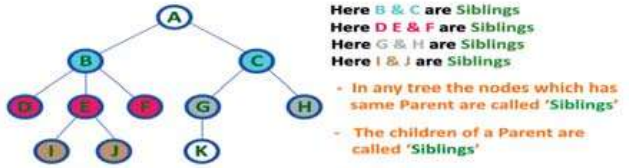


Figure 2.20.siblings

f. Leaf

In a tree data structure, the node which does not have a child is called a LEAF Node. In a tree data structure, the leaf nodes are also called a External Nodes or Terminal node.

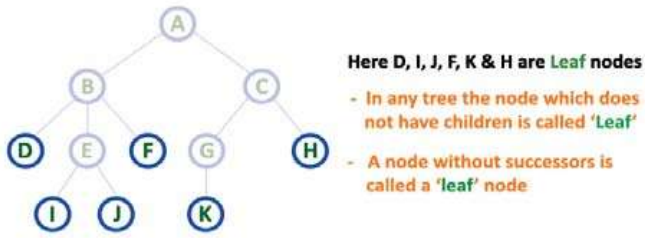


Figure 2.21: Leaf representation

g. Internal Nodes

In a tree data structure, the node which has at least one child is called **INTERNAL Node**. In a tree data structure, nodes other than leaf nodes are called **Internal Nodes**. **The root node is also said to be Internal Node** if the tree has more than one node. Internal nodes are also called as ‹**Non-Terminal**' nodes.

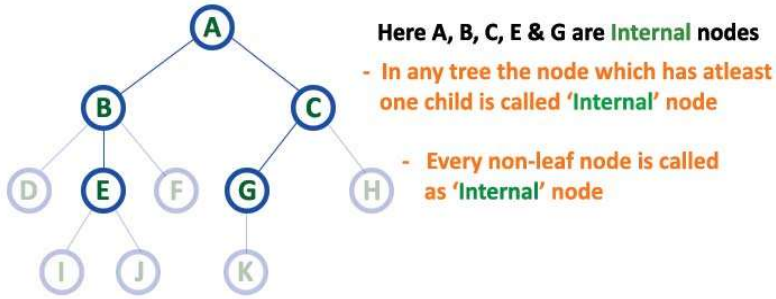


Figure 2.22: Internal node representation

h. Degree

In a tree data structure, the total number of children of a node is called as **DEGREE** of that Node. In simple words, the Degree of a node is total number of children it has. The highest degree of a node among all the nodes in a tree is called as ‹**Degree of Tree**'

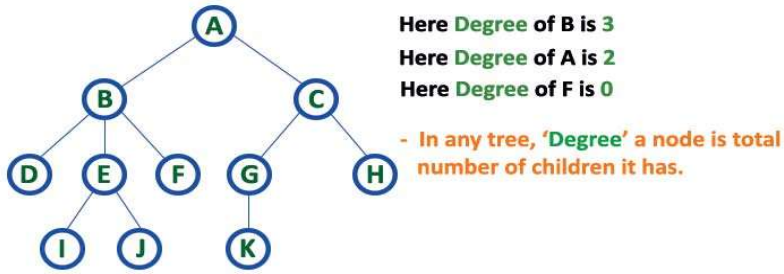


Figure 2.23: Degree representation

i. Level

In a tree data structure, the root node is said to be at Level 0 and the children of root node are at Level 1 and the children of the nodes which are at Level 1 will be at Level 2 and so on... In simple words, in a tree each step from top to bottom is called as a Level and the Level count starts with '0' and incremented by one at each level (Step).

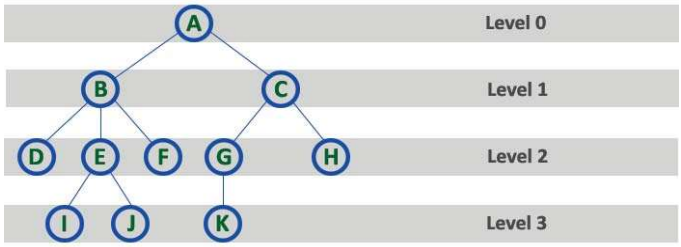


Figure 2.24: level representation

j. Height

In a tree data structure, the total number of edges from leaf node to a particular node in the longest path is called as **HEIGHT** of that Node. In a tree, height of the root

node is said to be **height of the tree**. In a tree, **height of all leaf nodes is '0'.**

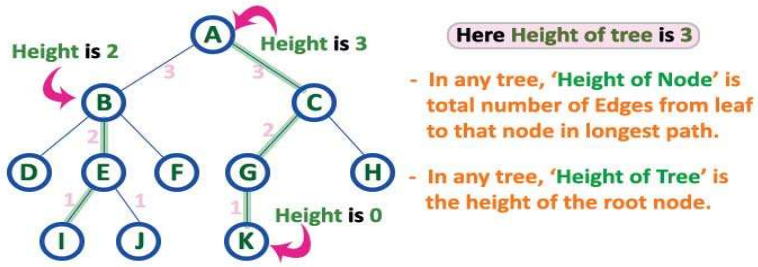


Figure 2.25.: Height representation

k. Depth

In a tree data structure, the total number of edges from root node to a particular node is called **DEPTH** of that Node. In a tree, the total number of edges from root node to a leaf node in the longest path is said to be **Depth of the tree**. In a tree, **depth of the root node is '0'.**

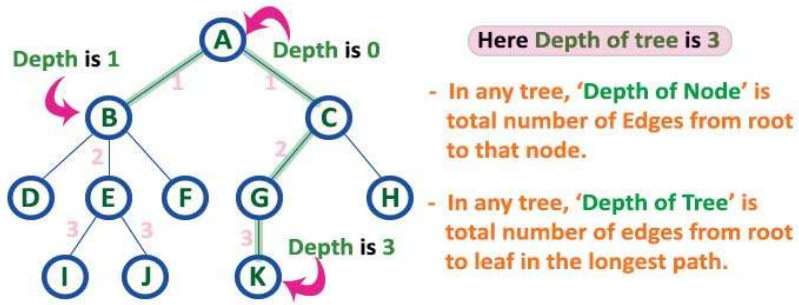


Figure 2.26.: Depth representation
Source: http://mitpolytechnic.ac.in/downlaods/09_knowledge-bank/02_computer/SEM-3/DS/Chap5_6.pdf

l. Path

In a tree data structure, the sequence of Nodes and Edges from one node to another node is called as **PATH** between those two Nodes. **Length of a Path** is total number of nodes in that path. In below example **the path A - B - E - J has length 4**.

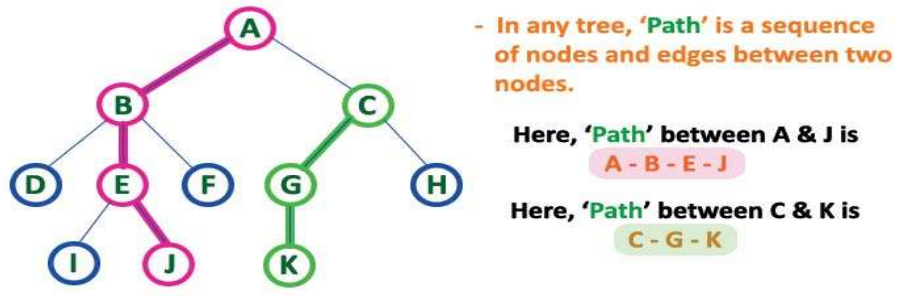


Figure 2.27: path representation

m. Sub Tree

In a tree data structure, each child from a node forms a sub tree recursively. Every child node will form a sub-tree on its parent node.

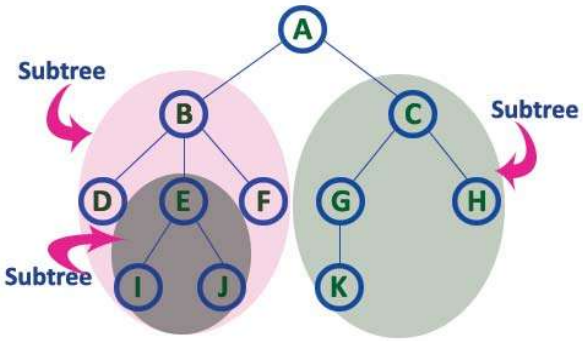


Figure 2.28: Sub tree representation

2.7.2 Binary Tree Representations

A binary tree data structure is represented using two methods. Those methods are as follows:

- Linked List of Binary Tree representation
- Array Representation

Let **T** be a Binary Tree. There are two ways of representing **T** in the memory as follow Linked

a. Linked List of Binary Tree representation

Consider a Binary Tree **T**. **T** will be maintained in memory by means of a linked list representation, which uses three parallel arrays; **INFO**, **LEFT**, and **RIGHT** pointer variable **ROOT** as follows. In Binary Tree, each node **N** of **T** will correspond to a location **k** such that:

1. **LEFT [k]** contains the location of the left child of node **N**.
2. **INFO [k]** contains the data at the node **N**.
3. **RIGHT [k]** contains the location of right child of node **N**.

- **Representation of a node:**

LEFT [k]	INFO [k]	RIGHT [k]

In this representation of binary tree, root will contain the location of the root **R** of **T**. If any one of the sub tree is empty, then the corresponding pointer will contain the null value if the tree **T** itself is empty, the **ROOT** will contain the null value.

Example

Consider the binary tree **T** in the figure of **Binary Tree**

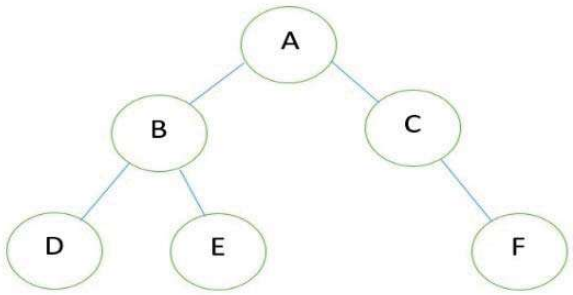


Figure 2.29: Binary Tree

Linked Representation of the Binary Tree

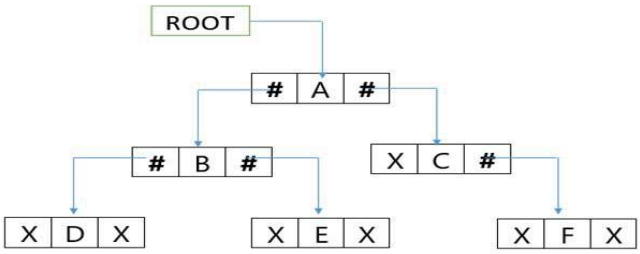


Figure 2.30.: Linked Representation of the Binary Tree

b. Array Representation of Binary Tree

Let us consider that we have a tree **T**. let our tree **T** is a binary tree that us complete binary tree. Then there is an efficient way of representing **T** in the memory called the sequential representation or array representation of **T**. This representation uses only a linear array TREE as follows:

1. The root **N** of **T** is stored in **TREE [1]**.
2. If a node occupies **TREE [k]** then its left child is stored in **TREE [2 * k]** and its right child is stored into **TREE [2 * k + 1]**.

For Example:

Consider the following Tree:

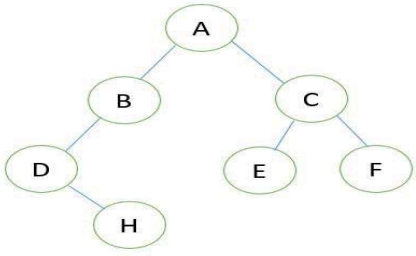


Figure 2.31: Array Representation of Binary Tree

Its sequential representation is as follow:

A	B	C	D	-	E	F	-	H		

2.7.3 Binary Tree Traversals

When we wanted to display a binary tree, we need to follow some order in which all the nodes of that binary tree must be displayed. In any binary tree displaying order of nodes depends on the traversal method.

Displaying (or) visiting order of nodes in a binary tree is called as Binary Tree Traversal.

There are three types of binary tree traversals.

- In - Order Traversal
- Pre - Order Traversal
- Post - Order Traversal

Traversal is a process to visit all the nodes of a tree and may print their values too. Because, all nodes are connected via edges (links) we always start from the root (head) node. That is, we cannot randomly access a node in a tree.

Generally, we traverse a tree to search or locate a given item or key in the tree or to print all the values it contains.

a. In-order Traversal

In this traversal method, the left sub tree is visited first, then the root and later the right sub-tree. We should always remember that every node may represent a sub tree itself.

If a binary tree is traversed **in-order**, the output will produce sorted key values in an ascending order.

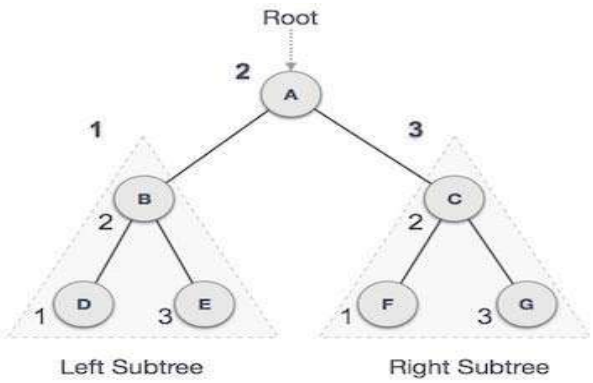


Figure 2.32: In-order Traversal

We start from **A**, and following in-order traversal, we move to its left subtree **B**. **B** is also traversed in-order. The process goes on until all the nodes are visited. The output of inorder traversal of this tree will be –

D → B → E → A → F → C → G

Algorithm:

Until all nodes are traversed –

Step 1 – Recursively traverse left subtree.

Step 2 – Visit root node.

Step 3 – Recursively traverse right subtree.

b. Pre-order Traversal

In this traversal method, the root node is visited first, then the left subtree and finally the right subtree.

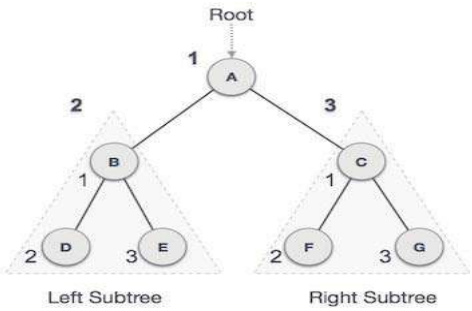


Figure 2.33: Pre-order Traversal

We start from **A**, and following pre-order traversal, we first visit **A** itself and then move to its left subtree **B**. **B** is also traversed pre-order. The process goes on until all the nodes are visited. The output of pre-order traversal of this tree will be –

A → B → D → E → C → F → G

Algorithm:

Until all nodes are traversed –

Step 1 – Visit root node.

Step 2 – Recursively traverse left subtree.

Step 3 – Recursively traverse right subtree.

c. Post-order Traversal

In this traversal method, the root node is visited last, hence the name. First we traverse the left sub tree, then the right sub tree and finally the root node.

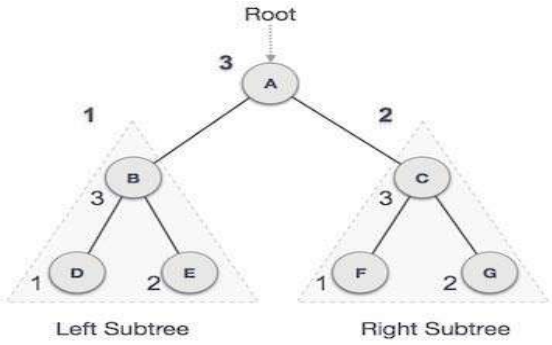


Figure 2.34.: Post-order Traversal

We start from **A**, and following Post-order traversal, we first visit the left sub tree **B**. **B** is also traversed post-order. The process goes on until all the nodes are visited. The output of post-order traversal of this tree will be −

D → E → B → F → G → C → A

Algorithm:

Until all nodes are traversed –

Step 1 – Recursively traverse left subtree.

Step 2 – Recursively traverse right subtree.

Step 3 – Visit root node.

Binary Search tree exhibits a special behavior. A node's left child must have a value less than its parent's value and the node's right child must have a value greater than its parent value.

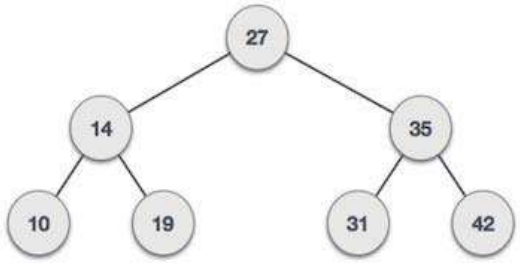


Figure 2.35: Tree Example

2.7.4. Basic Operations on binary tree

A Binary Search Tree (BST) is a tree in which all the nodes follow the below-mentioned properties

- The left sub-tree of a node has a key less than or equal to its parent node's key.
- The right sub-tree of a node has a key greater than to its parent node's key.

Thus, BST divides all its sub-trees into two segments; the left sub-tree and the right

sub-tree and can be defined as

left_subtree (keys) ≤ node (key) ≤ right_subtree (keys)

Representation

BST is a collection of nodes arranged in a way where they maintain BST properties. Each node has a key and an associated value. While searching, the desired key is compared to the keys in BST and if found, the associated value is retrieved.

Following is a pictorial representation of BST

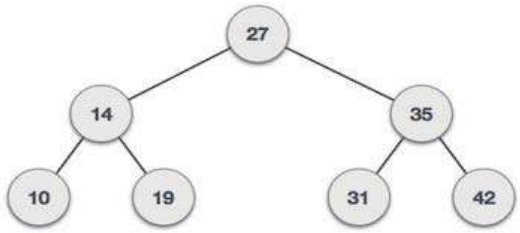


Figure 2.36.: ***Binary Search Tree***

We observe that the root node key (27) has all less-valued keys on the left sub-tree and the higher valued keys on the right sub-tree.

Basic Operations

Following are the basic operations of a tree –

- Search – Searches an element in a tree.
- Insert – Inserts an element in a tree.
- Pre-order Traversal – Traverses a tree in a pre-order manner.
- In-order Traversal – Traverses a tree in an in-order manner.
- Post-order Traversal – Traverses a tree in a post-order manner.

Node

Define a node having some data, references to its left and right child nodes.

```
struct node {
int data
struct node leftChild
struct node rightChild
};
```

Search Operation

Whenever an element is to be searched, start searching from the root node. Then if the data is less than the key value, search for the element in the left subtree. Otherwise, search for the element in the right subtree. Follow the same algorithm for each node.

Algorithm

```
struct node search(int data){
struct node current = root
  print "Visiting elements: "
while(current->data != data){
if(current != NULL){
      print current->data
//go to left tree
if(current->data > data){
        current = current->leftChild
//else go to right tree
else
        current = current->rightChild
end if//not found
if(current == NULL)
return NULL
      }
}
}
return current
}
```

Insert Operation

Whenever an element is to be inserted, first locate its proper location. Start searching from the root node, then if the data is less than the key value, search for the empty location in the left subtree and insert the data. Otherwise, search for the empty location in the right subtree and insert the data.

Algorithm

```
void insert(int data){
struct node tempNode =(struct node)
struct node current
struct node parent
```

```
  tempNode->data = data
  tempNode->leftChild = NULL
  tempNode->rightChild = NULL //if tree is empty
if(root == NULL){
    root = tempNode
}else{
    current = root
    parent = NULL
while(1){
      parent = current
//go to left of the tree
if(data < parent->data){
        current = current->leftChild
//insert to the left
if(current == NULL){
          parent->leftChild = tempNode
return
}
}//go to right of the tree
else{
        current = current->rightChild
//insert to the right
if(current == NULL){
          parent->rightChild = tempNode
return
}}}}}
```

2.8 Searching and sorting using complex data structure.

LEARNING ACTIVITY 2.4

Array A[10] is declared and initialized as: int A[10] = {10, 8, 2, 7, 3, 4, 9, 1, 6, 5};

a. Write down the steps to do the followings:

b. Search the value "7"

Definition

Searching is the process of finding the location of a given element in a set of elements.

Searching means to find whether a particular value is present in an array or not. If the value is present in the array, then searching is said to be successful and the searching process gives the location of that value in the array.

There are two simple approaches to searching:

Linear search: This method traverses a list sequentially to locate the search key.

Binary search: This method works on sorted lists by progressively making better guesses to find the location of a search key.

2.8.1 Linear search Algorithms

The linear search is also called sequential search. It is a simple method used for searching an array for a particular value. It works by comparing the value to be searched with every element of the array one by one in a sequence until a match is found.

Notice that the linear search is mostly used to search in an unordered list of elements

Linear Search Algorithm

linear search (a, n, item, loc)

Note:

- "a" is an array of the size n.
- "n" is the size of the array "a"
- "item" is the element to find in the array "a".
- "loc" is the index of an element in the array "a".

Begin
for i=0 to (n-1) by 1 do
if (a[i] = item) then
set loc=i
Print loc
exit
endif
setloc +1
endfor
end

2.8.2 Binary Search

Learning activity 2.5

Consider an array A[11] that is declared and initialized as:

int A[11] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

The value to be searched is VAL= 9

Analyze the 1st step and do the remaining one.

The algorithm proceeds in the following manner:

- BEG = 0, END = 10, MID = (0 + 10)/2 = 5

  Now, compare VAL = 9 and A[MID] = A[5] = 5
- A [5] is less than VAL. Therefore, we search for the value in the second half of the array. So, we assign the value of A[MID] to BEG.

- **Binary Search**

You search an array by repeatedly dividing the search interval in half. While dividing you begin with an interval covering the whole array. If the value of the search key is less than the item in the middle of the interval, narrow the interval to the lower half. Otherwise narrow it to the upper half. Repeatedly check until the value is found or the interval is empty. This kind of search is also called the dichotomy method or bisection method.

Schematically, it looks like the following:

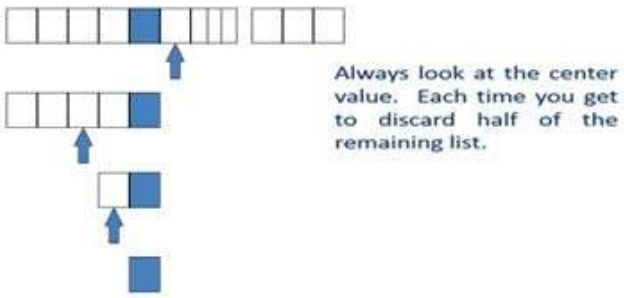


Figure 2.37: Process of Binary search or dichotomy method

- **The full binary search Algorithm is given down here.**

Binary search (a, n, item, loc)

Note:

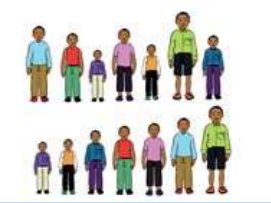
- "a " is an array of the size n.
- "n" is the size of the array "a"
- " item " is the element to find in the array "a".
- "loc" is the index of the element in the array "a".

```
Begin
set beg=0
set end=n-1
set mid=(beg+end)/2
while((beg<=end) and(a[mid]!=item) do
if(item<a[mid]) then
set end=mid-1
else
set beg=mid+1
endif
set mid=(beg+end)/2
endwhile
if(beg>end) then
set loc=-1
else
set loc=mid
endif
end
```

2.9 Sorting using complex data structure

Learning activity 2.5

Analyze the pictures below and answer the followings questions.



1. Describe the arrangement of these people in their rows
2. Compare the two rows of person

2.9.1. Definition:

Sorting is the process of arranging elements in some logical order either in ascending or descending order.

Methods for sorting

There are various methods for sorting in this section that we are going to discuss: Bubble sort, Insertion sort and Selection sort.

2.9.2. Bubble sort

Bubble Sort is a simple-minded algorithm based on the idea that we look at the list, and wherever we find two consecutive elements out of order, we swap them. This is done as follows: Repeatedly traverse the unsorted part of the array by comparing consecutive elements, and interchange them when they are out of order.

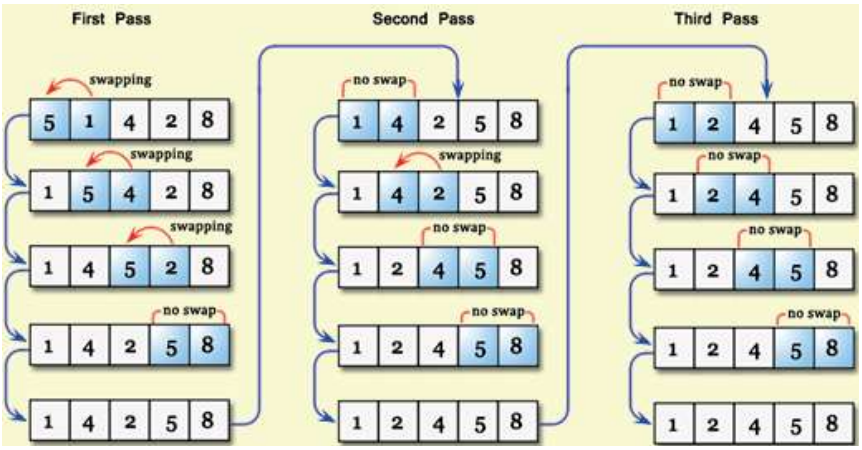


Figure 2.38.: Process of \|Bubble Sort

Source: https://www.jianshu.com/p/5bbfc8a191f5

Bubble Sort **Algorithm**

Begin
for k=1 to (n-1) by 1 do
for j=0 to (n-k-1) by 1 do
if(a[j]>a[j+1]) then
set temp=a[j]
set a[j]=a[j+1]
set a[j]=temp
endif
endfor
endfor
end

2.9.3. Insertion sort

The Insertion Sort is a comparison-based algorithm that builds a final sorted array one element at a time. It iterates through an input array and removes one element per iteration, finds the place if the element belongs in the array, and then places it there.

The illustration is given here below:

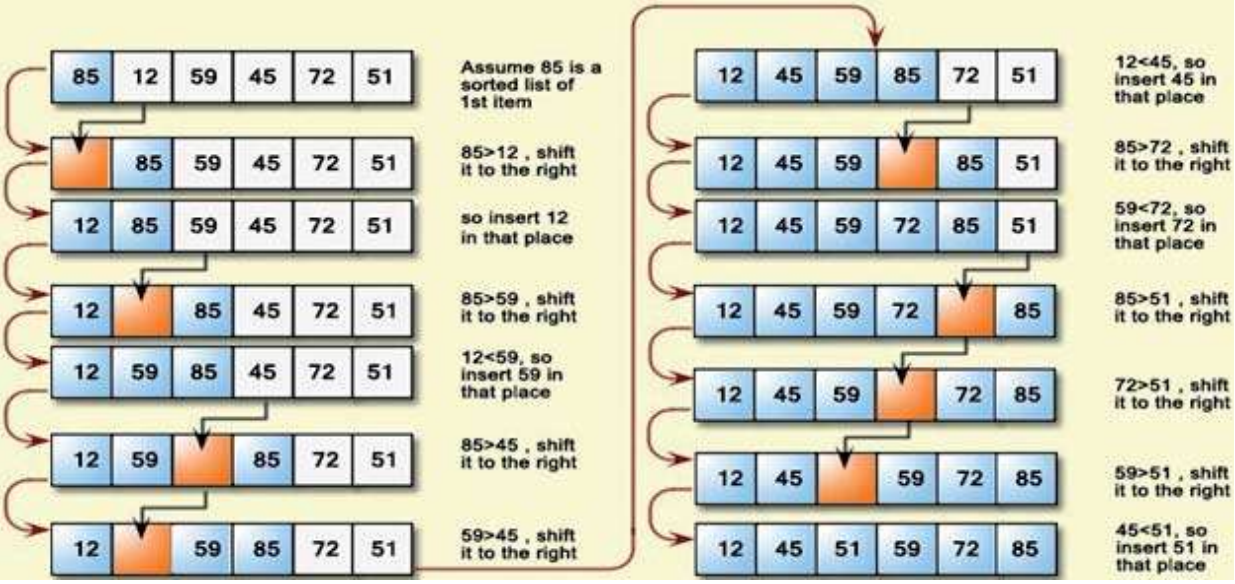


Figure 2.39.: process of insertion sort

Source: https://www.jianshu.com/p/5bbfc8a191f5

The Insertion Sort traverses the array and inserts each element into the sorted part of the list where it belongs. It involves pushing down the larger elements in the sorted part.

Insertion Sort Algorithm

Begin
For I = 1 to N-1
J = I
Do while (J > 0) and (A(J) < A(J - 1)
Temp = A(J)
A(J) = A(J - 1)
A(J - 1) = Temp
J = J - 1
End-Do
End-For
End

2.9.4. Selection sort

The Selection Sort is a simplicity sorting algorithm. Here are basic steps of selection sort algorithm:

i. Find the minimum element in the list
ii. Swap it with the element in the first position of the list
iii. Repeat the steps above for all remainder elements of the list starting at the second position.

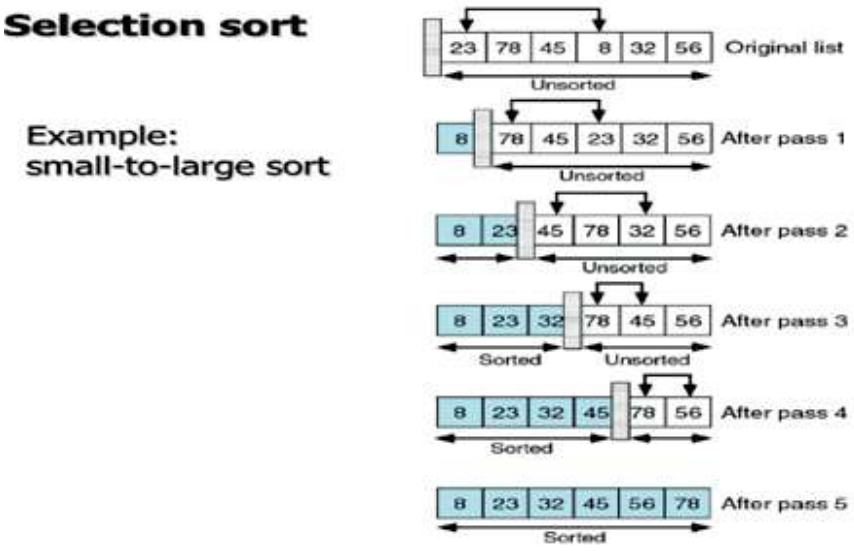


Figure 2.40.: process of selection sort

The idea of Selection Sort is that we repeatedly find the smallest element in the unsorted part of the array and swap it with the first element in the unsorted part of the array.

Selection Sort Algorithm

Begin
For I = 0 to N-1 do:
Smallsub = I
For J = I + 1 to N-1 do:
A(J) < A(Smallsub)
Smallsub = J
End-If
End-For
Temp = A(I)
A(I) = A(Smallsub)
A(Smallsub) = Temp
End-For
End

Application activity 2.4.

1. By using the bubble sort algorithm, write a program to sort an integer array of 10 elements in ascending order.
2. By using the sequential search algorithm, write an algorithm program to search for an element of an integer array of 10 elements.
3. Write a program to find the maximum and minimum value of an array.

2.10 Apply algorithm to solve complex mathematical functions

2.10.1. Understanding quadratic equation

LEARNING ACTIVITY 2.6.

Below you are given the algorithm and pseudo code to resolve quadratic equation: ax2+ b x+ c=0; where a, b and c are not equal to zero. Improve the algorithm and pseudo code to resolve quadratic equation since some of steps to resolve a quadratic equation are not given.

Algorithm:

- Start.
- Declare the required variables.
- Indicate the user to enter the coefficients of the quadratic equation
- Enter the input.
- Calculate the roots of quadratic equation using the proper formulae.
- Display the result.
- Stop. Pseudo Code
- Start.
- Input a, b, c.
- D — sqrt (b × b – 4 × a × c).
- X1 — (-b + d) / (2 × a).
- X2 — (-b - d) / (2 × a).
- Print x1, x2.
- Stop.

Quadratic equation is any mathematical equation having the form ax2+bx+c=0 where x represents an unknown, a, b, and c are constants.

The quadratic equation is a second degree polynomial equation since it only contains powers of x that are non-negative integers.

APPLICATION ACTIVITY 2.5

Draw the flow chart of the algorithm to solve quadratic equation.

2.10.2. Recursive function

Definition: A recursive function is a function that calls itself during its execution.

The Recursive function repeats itself several times, outputting the result at the end of each iteration. In this section we are going to discuss the following recursive functions:

- Summing a list of numbers;
- Factorial function;
- Fibonacci series function;
- Sorting;
- Palindrome.

- **Summing a list of numbers**

To calculate the sum of a list of numbers such as: [1, 3, 5, 7, 9], below is the pseudo code which is an iterative function that computes the sum of a list of numbers.

```
deflistsum(numList):
theSum = 0
fori in numList:
theSum = theSum + i
returntheSum
print(listsum([1,3,5,7,9]))
```

The function uses an accumulator variable (thesum) to compute a running total of all the numbers in the list by starting with 00 and adding each number in the list

APPLICATION ACTIVITY 2.6.

1. Analyze the below pseudo codes and give the general formula of the function below.

   If n = 5 sum(n) = 1+2 +3 +4 +5 =15

   If n=6 sum(n) = 1 +2 +3 +4 +5 +6= 21

2. Write the algorithm and draw the flow chart to calculate the sum of numbers in a list.

2.10.3 Factorial function

LEARNING ACTIVITY2.8

Analyze the pseudo code below and give the general formula.

```
double factorial (int n )
{
double  fac1;
fac1 = 1;
for (inti =1; i<= n ; i ++)
fac1 =fac1*i;
return ( fac1);
}
```

In mathematics, the factorial of a non-negative integer n, denoted by n! is the product of all positive integers less than or equal to n.

Notice that the value of 0! is 1, according to the convention for an empty product.

The factorial function (symbol: !) means to multiply a series of descending natural numbers. Examples:

$4! = 4 \times 3 \times 2 \times 1 = 24$

$7! = 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 5040$

$1! = 1$

APPLICATION ACTIVITY 2.7

1. Analyse the pseudo codes below and answer following questions:

Pseudo Code A

```
double factorial( int n )
{
double  fac1;
fac1 = 1 ;
for ( inti =1; i<= n ; i ++)
fac1 =fac1 *i;
return ( fac1);
 }
```

```
        Pseudo Code B
double factorial( int n )
{
 if ( n ==1 ) // base case
 return 1;
  return n * factorial( n  - 1 );
}
```

i. Predict the result of each pseudo code
ii. What do you think of each concept used?
iii. Are they representing the same algorithm? If yes, which one is not time consuming and space consuming? Explain why.

2. Draw the flow chart to calculate factorial number.

2.10.4 Fibonacci series function

LEARNING ACTIVITY 2.9

F (0) = 0

F (1) = 1

F (2) = F (2-1) + F (2-2)

= F (1) + F (0) = 1 + 0 = 2

1. Compute F (3), F (4), F (5),
2. Analyze the above pseudo codes and deduct the general formula

In mathematics, the Fibonacci numbers are integer numbers in the following sequence: 0, 1, 1, 2, 3, 5, 8..., this series of integers is called the Fibonacci sequence. Fibonacci sequence is characterized by the fact that every number after the first two is the sum of the two preceding one. It means that except for the first two terms of the sequence every other term is the sum of the previous two terms.

The pseudo code of Fibonacci function

```
Fibo(n)
Begin
if n <= 1 then
Return n;
else
Return Call Fibo(n-1) + Call Fibo(n-2);
endif
End
```

APPLICATION ACTIVITY 2.8..

Write the algorithm and draw the flow chart to calculate Fibonacci Series.

2.10.5. Palindrome function

LEARNING ACTIVITY 2.10..

Investigate and find at least 5 Kinyarwanda's words which do not change when its letters are reversed.

INSTRUCTIONS:

- Extract the letters from the word.
- Reverse their order.
- Paste them back together into a word.
- Compare with the original word.

The algorithm which would get out any amount palindrome numbers on screen.

Let take: 300003 310013 320023 330033 340043 350053.

Findings: written algorithm how to check if it is a palindrome number or not.

Here is algorithm for how i check if its palindrome or not:

```
BEGIN
   int n, num, dig, rev = 0
   PRINT "Insert number"
   READ num
   n = num
  while (num != 0)
   {
      dig = num % 10
      rev = (rev * 10) + dig
      num = num / 10
   }
   if (n == rev)
   PRINT "This is palindrome", rev
   else
   PRINT "This is not palindrome " rev
    END IF
END
```

A palindrome is a word that is spelled the same from both ends.

For example: anna, ifi, isi, and malayalam are palindromes.

The table below gives two basic ways of determining whether a word is a palindrome.

Pseudo code:	Detailed Algorithm:
Input a String	Step 1: Input S (string)
Initialize Len to zero , Flag to zero	Step 2: Len = 0 , Flag =0
While String[Len] is not equal to NULL	Step 3: While (S[Len] != NULL)
Increment Len	Len++
Initialize I to zero , J to Len-1	Step 4: I = 0 , J = Len-1
While I is less than (Len/2)+1	Step 5: While (I < (Len/2)+1)
If String[I] equal to String[J]	If (S[I] == S[J])
Flag=0	Flag=0
else	else
Flag=1	Flag=1
Increment I , Decrement J	I++ , J–
If Flag equal to zero	Step 6: If (Flag == 0)
Print Key Is a Palindrome	Print Key Is a Palindrome
else	else
Print Key Is Not a Palindrome	Print Key Is Not a Palindrome
Stop	Step 7: End

APPLICATION ACTIVITY 2.9..

From the above pseudo code and detailed algorithm draw the flowchart that check whether a given string is a palindrome or not

2.10.6. Euclid's GCD(Greatest common denominator)

LEARNING ACTIVITY 2.11.

Given int a=210 and int b=45;

Find the greatest common divisor of two positive integers a and b.

Write down all steps to find the greatest common denominator.

Suggest the algorithm to find the greatest common denominator of two numbers.

Compare with the original word.

LEARNING ACTIVITY 2.12.

Analyse the following pseudo code and answer to the question below.

1. Start 2. read two numbers a,b 3. set n=b,d=a 4. r=a%b 5. while r!=0 5.1 set n=d,d=r 5.2 r=n%d 6. setgsc=d 7. lcm=(a*b)/gcd 8. displaygcd and lcm 9. stop	begin Let a, b denote the numerator and denominator, respectively. Let 'gcd' denote the Greatest Common Divisor of both a and b. *If (a==0 || b==0)* *return gcd = 1;* *else* *{* *r = (a%b);* *while (r < 0 || r > 0)* *{* *a = b;* *b = r;* *r = (a%b);* *}* *gcd = b;* *}* *return gcd;* *end*

c. What the 2 pseudocodes are giving as results?

d. Replace a by 24 and b by 16. What is the results of the two pseudo codes?

- **Greatest Common Denominator**

The greatest common denominator, or GCD, between two numbers is the greatest integer that divides both given numbers.

- **Euclidean Algorithm**

The Euclidean Algorithm is pretty straight forward: given two numbers, repeatedly replace the larger number with the greater number mod the lesser number. Keep repeating the step until one of the two numbers reaches zero. The other number will then be the greatest common denominator (or greatest common divisor), the GCD.

1. Divide m by n and let r be the remainder.
2. If r is 0, n is the answer; if r is not 0, continue to step 3.
3. Set m = n and n = r. Go back to step 1.

2.11 END UNIT ASSESSMENT

1. Examine the following:

```
Double[][] values =
 {{1.2, 9.0, 3.2},
  {9.2, 0.5, 1.5, -1.2},
  {7.3, 7.9, 4.8}} ;
```

what is in values[2][1] ?

2. You want to create a table that looks like:

12	-9	8
7	14	
-32	-1	0

3. Which of the following will work

A

```
double[][]  table =
  { 12, -9, 8,
    7, 14,
    -32, -1, 0} ;
```

B

```
double[][] table =
  { {12, -9, 8},
    {7, 14, 0},
    -32, -1, 0} };
```

```
double[][] table =
  { {12, -9, 8}
    {7, 14}
    {-32, -1, 0} };
```

C

```
double[][] table =
  { {12, -9, 8},
    {7, 14},
    {-32, -1, 0} };
```

D

4. Write an algorithms that allows a user to enter 4 elements for a 2x2 array and print their product.
5. A linear list of elements in which deletion can be done from one end (front) and insertion can take place only at the other end (rear) is known as a ?
   a. Queue
   b. Stack
   c. Tree
   d. Linked list

6. Write a Fibonacci series algorithm of any given number entered through the keyboard
7. Write algorithm where the user is asked to input a number and print it's factorial
8. For each of the algorithm designed in the different activities, write their corresponding programmes in C++ languages and verify the results.

UNIT 3

INTRODUCTION TO COMPUTER NETWORK

UNIT 3: INTRODUCTION TO COMPUTER NETWORK

Key Unit Competency:

To be able to explain principles, standards and purposes of computer network

Read the following scenario and answer to the asked questions.

In the school computer labs, choose a classmate and send him/her an email that has "Computer Network "as the email subject and the email body contains names of previous units already covered in senior 5 computer science. Ask him/her to reply immediately to your email.

After receiving the reply, answer the following questions:

1. List all possible elements that make this communication possible
2. What devices are used to facilitate sending email successfully?
3. Discuss what made possible the sending and receiving of emails
4. Do you like using email?
5. How do you value the factor of sending and receiving an email
6. Describe what you see in the photo below?



3.1. Fundamental of computer Networ

3.1.1. Computer Network Definition

ACTIVITY 3.1

Observe computers in the computer lab and answer the following question?

1. Describe what you see?
2. What are transmission media used to connect computers in the computer labs?
3. Are all devices connected?

A computer network is a system of interconnected computers for the purpose of sharing data and resources.

Computers on a network are called nodes. Nodes can include hosts such as personal computers, phones, servers as well as networking hardware. Two devices are networked together when one device is able to exchange information with the other device. The computer can be connected to another in the two ways.

Wired network: Computers are connected using cable media. Most commonly Ethernet Cable, coaxial cable and optic fiber.

Wireless network: Computers are connected using wireless media. Radio waves are used in wireless mode.

3.1.2 Properties of Computer Network

- **Easy Sharing of Resources:** Computers are able to share various resources easily over a network. Shared resources can be Internet, files, printer, storage and others.
- **Performance:** It is achieved by measuring the speed of data transmission with number of network users, connectivity used and the software used. The commonly measured qualities in the network performance are **Bandwidth and Latency.**
- **Reliability:** It means that computer network provides assurance of the delivery of data to the intended recipient.
- **Scalability:** The possibility of adding new computer without affecting the network performance.
- **Security:** computer network must be secured for the benefit of the user and data protection. The security is achieved by protecting data from unauthorized access.

- **Quality of Service (QoS):** Quality of Service refers to the mechanism that **manage congested network traffic.**
- **Fault tolerant:** A fault tolerant network limits the impact of hardware or software failure and recovers quickly when a failure occurs

APPLICATION ACTIVITY 3.2

1. Investigate what happen when a shared resource is disconnected from the computer network?
2. Imagine other areas where computer network is used?

3.1.3 Advantages and Disadvantage of Computer Networking

The common advantages of computer network

Activity 3.2

1. Discuss the disadvantages of using computer network within your school?
2. In class of senior 5 MPC, Computer Science Teacher decides to share notes for students to use on our school network. The mathematics teacher decides to print notes of 100 pages and distributes to each member of senior 5 MPC class.
   i. Evaluate the cost impact for both teachers
   ii. Both teachers, whom save money in giving notes.

The common advantages of computer network

1. **Enhanced communication and availability of information:** It allows access to a vast amount of useful information.

   Example: Population data, newsletters, online businesses, contents, Applications.

2. **Allow resource sharing:** Fewer resources are needed when an organization uses a computer network.

   Example: only one printer is needed instead of buying a printer for each office.

3. **File sharing made easy:** computer network allows people to share files, which helps to save more time and effort.

   Example: A teacher can share homework to all students through school network

4. **Improve storage capacity:** since you are going to share information, files and resources to other people, you have to ensure that all data and content are properly stored in the system.

5. **Cost efficiency:** on computer network, you can share software license installed on the server and can then be used by various workstations.
6. **Security of information and resources:** users cannot see other users' files unlike on stand-alone machines.
7. **Backup of data is easy** as all the data is stored on the file server.

The common disadvantages of computer networking

1. **Lack of independence:** people rely on computer network and when the system is down, people get stack. Most of organizations depend on the computer networks.
2. **Security issues:** huge number of people use a computer network to get and share their files and resources, a certain user's security would be always at risk. Viruses can spread to other computers throughout a computer network. There is a danger of hacking, particularly with wide area networks. Security procedures are needed to prevent such abuse, Examples: The use of Antivirus and firewall.
3. **Lack of robustness:** computer network's main server breaks down, the entire system would become useless

APPLICATION ACTIVITY 3.2

1. Investigate what happen when a shared resource is disconnected from the computer network?
2. Imagine other areas where computer network is used?

3.1.4 Types of computer networks

Activity 3.3

With two or more Bluetooth enabled devices available in the classroom:

1. Turn on the Bluetooth feature on both devices
2. Share and receive files such images and documents between the devices
3. Turn off Bluetooth feature in the two devices.

A computer network is classified basing on the size of the area covered, the number of users connected and the number & types of services available.

The followings are type of computer networks:

1. Personal Area Network (PAN)

A personal area network (PAN) is a network that connects devices, such as mice, keyboards, printers, Smartphone, and tablets within the range of an individual person. PAN has connectivity range up to 10 meters. PAN may include wireless computer keyboard and mouse, Bluetooth enabled headphones, wireless printers and TV remotes. All of these devices are dedicated to a single host and are most often connected with Bluetooth technology.

Bluetooth is a wireless technology that enables devices to communicate over short distances. A Bluetooth device can connect up to seven other Bluetooth devices.

2. Local Area Network (LAN)

Traditionally, a LAN is defined as a network that covers a small geographical area. However, the distinguishing characteristic for LANs today is that they are typically owned by an individual, such as in a home or small business, or wholly managed by an IT department, such as in a school or corporation. This individual or group enforces the security and access control policies of the network. LAN uses Ethernet IEEE 802.3 as its standard.

A Wireless LAN (WLAN) is a LAN that uses radio waves to transmit data between wireless devices. In a traditional LAN, devices are connected together using copper cabling. In some environments, installing copper cabling might not be practical, desirable, or even possible. In these situations, wireless devices are used to transmit and receive data using radio waves. As with LANs, on a WLAN, you can share resources, such as files, printers, and Internet access. WLAN uses Ethernet IEEE 802.11 as its standard.

3. Metropolitan Area Network (MAN)

A metropolitan area network (MAN) is a network that spans across a large organization like campus or a city. The network consists of various buildings interconnected through wireless or fiber optic backbones.

A backbone is the part of the computer network infrastructure that interconnects different LAN networks and provides a path for exchange of data between these different networks

The communication links and equipment are typically owned by a network service provider who sells the service to the users. A MAN can acts as a high-speed network to enable sharing of regional resources.

4. Wide Area Network (WAN)

A WAN connects multiple networks that are in geographically separated locations.

The distinguishing characteristic of a WAN is that it is owned by a service provider. Individuals and organizations contract for WAN services. WAN network provides connectivity to MANs and LANs. The most common example of a WAN is the Internetwork or Internet in short. The Internet is a large WAN that is composed of millions of interconnected networks.

Example: Kigali and Nairobi networks are connected through the Internet

APPLICATION ACTIVITY 3.3

1. Discuss the role ISP services for LANs?
2. Do an observation in the school computer lab and categorize the computer network type available at your school? Justify your answer.
3. Using clear examples, Distinguish Personal Area Network and Local Area Network?
4. Using an arrow match the following in Group A with their corresponding in Group B

Group A	Group B
a. Radio waves	1. Qualities of Computer network Performance
b. Millions of interconnected network	2. Disadvantages of Computer network
c. Sharing resources	3. Wireless
d. Hacking attacks	4. WAN
e. Bandwidth and Latency	5. Advantages of Computer network

3.2 Computer Network Concept and Technology

ACTIVITY 3.4

You are browsing a website on the internet while the speed is okay and suddenly you find that the page is loading slowly. Investigate the reason behind slowness?

3.2.1 Network metrics

Network metrics are defined as standards of measurement by which efficiency, performance, progress, or quality of a plan, process can be assessed. Network metric are used by a router to make routing decisions.

When data is sent over a computer network, it is broken up into small chunks called **packets.** Each packet contains source and destination address information. Packets are sent across a network one bit at a time.

a. Bandwidth

Bandwidth is the amount of data that can be transmitted in a fixed amount of time. Bandwidth is measured in the number of bits that can be sent every second.

The following are examples of bandwidth measurements:

- b/s - bits per second
- kb/s - kilobits per second
- Mb/s - megabits per second
- Gb/s - gigabits per second

b. Latency

Latency is the time between requesting data and receiving data. More simply put, the time it takes to establish a connection between your computer and the server hosting the website you requested. The important thing to take away here is that latency is not speed. Data is delayed by network devices and cable length. Network devices add latency when processing and forwarding data.

c. Throughput:

Throughput is the actual rate at which information is transferred. It measures the amount of completed work against time consumed and may be used to measure the performance of a processor, memory and/or network communications.

d. Error rate:

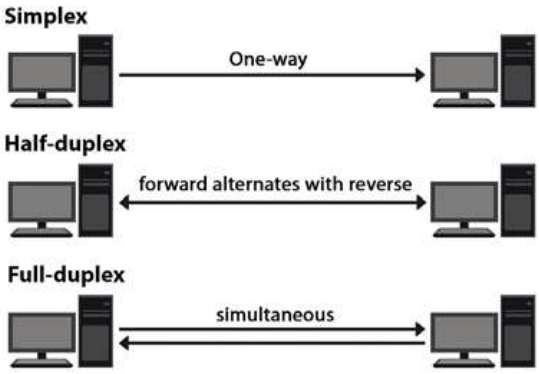
Error rate is the number of corrupted bits expressed as percentage or fraction of the total sent.

e. Jitter:

Jitter is variation in packet delay at the receiver of information. For measuring the capacity of all of these metrics above, we focus on capacity of message and this one change into different form depending on the network devices where this message is located.

3.2.2 Data Transmission model

The term transmission mode is used to define the direction of signal flow between two linked devices. The data that is transmitted over the network can flow using one of three modes: simplex, half-duplex and full-duplex.



Picture 3.1. Transmission modes: simplex, half-duplex and full-duplex

1. **Simplex:** it is a single one-way transmission. In a **simplex transmission mode**, the communication between sender and receiver occurs only in one direction. That means only the sender can transmit data, and receiver can receive that data. The receiver cannot transmit any information back to the sender.

   The keyboard to monitor transmission is an example of simplex transmission mode. Other examples are communication between a computer and a printer, listening to the radio and the signal that is sent from a TV station to your home TV.
2. **Half-Duplex:** data flows in one direction at a time. In half-duplex, the channel of communications allows alternating transmission in two directions, but not in both directions simultaneously. An Example of Half-duplex is the Talkie- Walkie used by the police.
3. **Full-Duplex:** data flows in both directions at the same time. In a **full duplex transmission mode**, the communication between sender and receiver can occur simultaneously. Sender and receiver both can transmit and receive simultaneously at the same time.

Basis for Comparison	Simplex	Half Duplex	Full Duplex
Direction of Communication	Communication is unidirectional.	Communication is two-directional but, one at a time.	Communication is two directional and done simultaneously.
Send/Receive	A sender can send data but, cannot receive.	A sender can send as well as receive the data but one at a time.	A sender can send as well as receive the data simultaneously.

Performance	The half-duplex and full duplex produces better performance than the Simplex.	The full duplex mode produces higher performance than half duplex.	Full duplex has better performance as it doubles the utilization of bandwidth.
Example	Keyboard and monitor.	Walkie-Talkies, internet chart	Telephone.

Table 3.1: comparison of network transmission modes

A telephone conversation is an example of full-duplex communication. Both people can talk and be heard at the same time.

APPLICATION ACTIVITY 3.4

1. Examine 2 reasons that cause latency in computer network.
2. What is your school download bandwidth and upload bandwidth?
3. Contrast upload bandwidth and download bandwidth?
4. Categorize following devices according to their transmission modes
   a. TV
   b. Radio
   c. Walkie -Talkie
   d. Phone
   e. Computer

3.2.3. Internetwork (Internet, Intranet, Extranet)

ACTIVITY 3.5

Using computer network, an organization's employees can access data when they are inside the organization buildingand when they are physically outside. Investigate technologies that can be used to allow employees inside and outside to access organization's data.

A network of networks is called an internetwork, or simply the **Internet**. The Internet, extranets, and intranets all rely on Transport Control Protocol / Internet Protocol (TCP/IP). However, they are different in terms of the levels of access they allow to various users inside and outside the organization and the size of the network.

1. **An Intranet** is a private computer network that uses Internet Protocol to securely share any part of an organization's information or operational systems within that organization. Only users inside the organization are only allowed to access it.
2. **An Extranet** is a private network that uses Internet protocols, network

connectivity. An extranet can be viewed as part of a company's intranet that is extended to users outside the company, the connectivity is made possible by the Internet.

3. **The Internet** is a global system of interconnected computer networks that use the standard Internet Protocol suite (TCP/IP) to serve billions of users worldwide.

The difference between the Internet and Extranets is that while the extranet allows limited access to non-members of an organization, the Internet generally allows everyone to access all network resources.

APPLICATION ACTIVITY 3.5

General supply Ltd. sell various services online and offline. It has various partners such as suppliers, sellers, buyers and clients in different Districts of Rwanda. Each partner wants to access the General supply data from its servers located at the main office in Ruhango District.

Examine in details the appropriate technology for each user to access the General supply Ltd. data to avoid the user location problem

3.3 Local Area Networks (LANs)

ACTIVITY 3.6

By definition, computer network is said to be a group of computers connected together and share resources. Using your school environment, observe followings:

1. The range covered by your school network?
2. The number of users on your school network?
3. Resources shared on your school network?

3.3.1 Definition

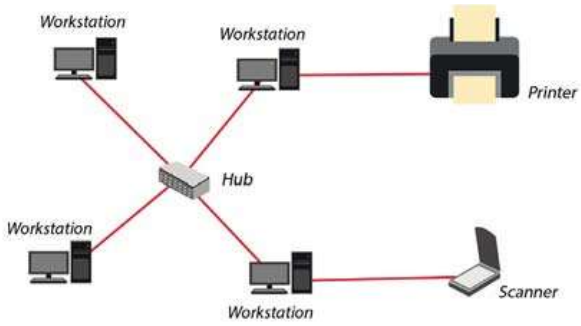
Local Area Networks (LANs) are networks usually confined to a geographic area, such as a single building or a college campus. LANs can be small, linking as few as three computers, but often link hundreds of computers used by thousands of people.

3.3.2 LAN Categories

1. Peer-to-Peer Network

A peer-to-peer (P2P) network is created when two or more computers are connected and share resources without going through a separate server computer. In a peer-to-peer network, there is no hierarchy among the computers, nor are there any dedicated servers.

Each device on the network, also called a client, has equivalent capabilities and responsibilities. A user is responsible for its own resources and can decide which data and devices to be shared with other computers. Because individual users are responsible for the resources on their own computers, the network has no central point of control and no central administration. Peer-to-peer networks work best in environments with ten or fewer computers.



Picture 3.2: Peer-to-Peer network

Advantages of peer to peer network are as follows:

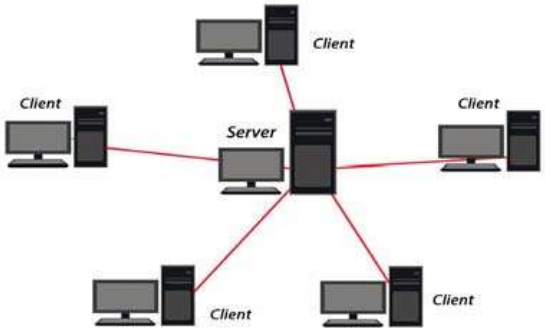
- The main advantage of peer to peer network is that it is easier to set up
- In peer-to-peer networks all nodes are act as server as well as client therefore no need of dedicated server
- The peer to peer network is less expensive
- Peer to peer network is easier to set up and use this means that you can spend less time in the configuration and implementation of peer to peer network.

Disadvantages of peer to peer network

- A computer can be accessed anytime
- Network security has to be applied to each computer separately
- Backup has to be performed on each computer separately
- No centralized server is available to manage and control the access of data.

2. Client-Server Network

In a client-server network, the client requests information or services from the server and the server provides the requested information or service to the client. Servers on a client-server network commonly perform some of the processing work for client machines.



Picture 3.3: Client-Server Network

In a client-server network, resources are controlled by a centralized network administration. The network administrator implements data backups and security measures. The network administrator also controls user access to the server resources.

Note: In a home or small business, a single server can run multiple types of server software, it may be necessary for one computer to act as a file server, a web server, and an email server. A client computer can also run multiple types of client software. There must be client software for every service required. With multiple client software installed, a client can connect to multiple servers at the same time.

Advantages of Client-Server Network

- **Centralization of control:** Access, resources and integrity of the data are controlled by the dedicated server so that a program or unauthorized client cannot damage the system.
- **Scalability:** You can increase the capacity of clients and servers separately. Any element can be increased (or enhanced) at any time, you can add new nodes to the network (clients or servers).
- **Easy maintenance:** Distribute the roles and responsibilities to several standalone computers, you can replace, repair, upgrade, or even move a server, while customers will not be affected by that change (or minimally affect).

Disadvantage of Client Server Networks

- There is a reliance on the central server, if it fails, no work can be done
- A network manager is required and this costs money
- The server costs money, as does the network operating system
- Servers are powerful, thus expensive
- Lots of network traffic.

Advantages of Client Server Networks over Peer to Peer Networks

- **Centralization:** Unlike Peer to Peer, where there is no central administration, Client Server Networks have a centralized control.
- **Proper Management :** All the files are stored at the same place. Therefore, management of files becomes easy and it is easier to find files.
- **Back-up and Recovery possible:** As all the data is stored on server it is easy to make a back-up of it. Also, in case of some break-down if data is lost, it can be recovered easily and efficiently. While in peer to peer network, we have to take back-up at every workstation.
- **Upgrade and Scalability in Client-server set-up:** Changes can be made easily by just upgrading the server. Also new resources and systems can be added by making necessary changes in server.
- **Accessibility:** From various platforms in the network, server can be accessed remotely.
- Security: Rules defining security and access rights can be defined at the time of set-up of server.

Disadvantages of client server network over the Peer to peer network

- Congestion in Network: Too many requests from the clients may lead to congestion, which rarely takes place in Peer to Peer network. Overload can lead to breaking-down of servers. In peer-to-peer, the total bandwidth of the network increases as the number of peers increase.
- Client-Server network is not as robust as a Peer to Peer network and if the server fails, the whole network goes down.
- Cost: It is very expensive to install and manage client server network.

APPLICATION ACTIVITY 3.6

Respond the following questions by True or False and justify your answer

- Peer to Peer network is only implemented by computers which run same operating system.
- In Server-Client network, Server respond to service only requested by the Client.
- Limit number of hosts in the Peer to Peer network are 10 computers.
- Client requests only the service available on the server in Server-Client network.
- Peer to Peer network is more secure than Server-Client network.
- There is a dedicated computer in Peer to Peer network.
- In Client- Server network, both client and server must run the same Operating System.

3.4. Physical Components

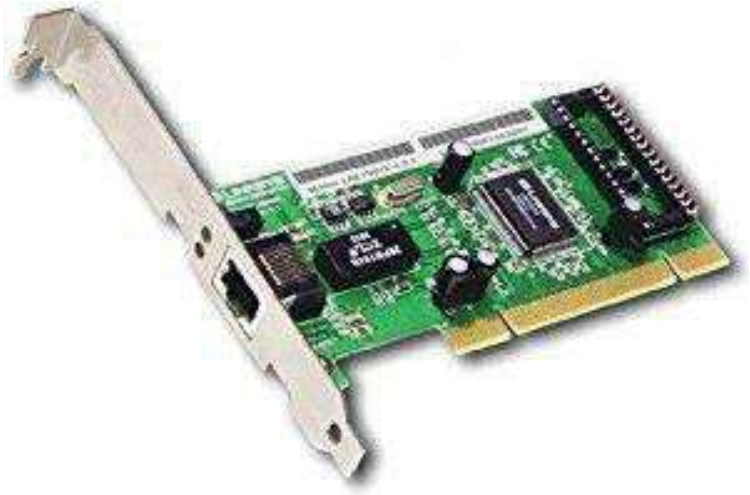
ACTIVITY 3.7

In the computer lab observe devices that are directly participating in the network. Examine the role of each device you have identified.

3.4.1. Definition of network device

A network host or a node is a computer or any other device that is directly connected to a computer network. A network host may offer information resources, services, and applications to users.

i. Network Interface Card (NIC)



Picture 3.4: Network Interface Card
Source: http://www.alpagut.com/ImageGallery.aspx?PID=1196&t=1

Network Interface card is a component that allows the computer to communicate across a network. This component is frequently built into the motherboard of today's computers, but it can also be a separate card for use in a PCI slot.

Network Interface cards can be either wired or wireless. However, some cards do support both wireless and wired networking

APPLICATION ACTIVITIES 3.7

NIC Card installation in a desktop computer:

Step 1: Shut down your Computer and disconnect it from the power source

Step 2: Remove the case cover. Then remove the cover of the available slot.

Step 3: Install NIC in proper slot and secure it

Step4: Replace the case cover.

Note: A wireless NIC has an antenna connected to the back of the card or attached with a cable so that it can be positioned for the best signal reception. You must connect and position the antenna.



Picture 3. 5: Wireless Network Interface Card

Source: https://www.ictlounge.com/html/network_hardware.htm

ii. Media Access Control Address

Media Access Control Address (MAC address) of a device is a unique identifier assigned to network interfaces for communications at the data link layer of a network segment. MAC address is a physical address of Network Interface Card. In other words MAC addresses are linked to the hardware of network adapters. A MAC address is given to a network adapter when it is manufactured.

Example of a MAC address: *00:0a:95:9d:68:16*.

iii. Modem

A modem is a hardware device that enables a computer to send and receive data over a telephone line or a cable or satellite connection. Modem is used to transmit digital information via analog systems. The word "modem" is derived from the term "**Mo**dulator - **Dem**odulator."

The essential functions of a modem are:

- **Modulate:** an analog carrier signal to carry digital information, it means to convert the analog signal to digital signal.
- **Demodulate:** a similar signal so as to decode the digital information from the analog carrier signal and it means to convert the same signal back to the analog signal then transmitted through telephone line.

They are two types of modems:

Internal modem which are circuit boards that plug into a computer's motherboard and external modems which are discrete units housed in a separate case. Typically, an external modem is connected to the telephone line and the computer via cables or USB.